



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 05-Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	1ro Software - Paralelo “ B”
Alumnos participantes:	Loor Velez Jhon Alejandro Bravo López Jordan Samuel Cajiao Valdivieso Paulo Alessandro Balarezo Pérez Diego Sebastián Pomaquero Chango Katherine Soledad
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas utilizando clases, métodos y manejo de archivos para registrar, almacenar y manipular información mediante la creación, lectura y escritura de ficheros en C ++ y Java.

Específicos:

- Implementar programas que permitan guardar y recuperar información utilizando archivos de texto en C ++ y Java.
- Aplicar programación orientada a objetos mediante clases, objetos y métodos para organizar mejor el código y disminuir errores en el sistema.
- Utilizar los conocimientos adquiridos en la guía APE-05 para contribuir al desarrollo del aplicativo web del proyecto final, mejorando el manejo y almacenamiento de información dentro del sistema.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma grupal.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java y C ++ cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:



- ☐ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

2.6 Actividades por desarrollar

- Analizar los requerimientos de los ejercicios planteados en la guía práctica APE-05.
- Diseñar programas utilizando clases, objetos y métodos en C++ y Java.
- Implementar la creación, lectura y escritura de archivos de texto para almacenar información.
- Aplicar validaciones y operaciones necesarias dentro de cada sistema desarrollado.
- Aplicar los conocimientos adquiridos en el desarrollo del aplicativo web del proyecto final para mejorar el almacenamiento y manejo de información dentro del sistema.

2.7 Resultados obtenidos

Se logró desarrollar programas en C++ y Java aplicando programación orientada a objetos y manejo de archivos para registrar, guardar y mostrar información correctamente. Los sistemas desarrollados permitieron realizar operaciones de creación, lectura y escritura de ficheros utilizando métodos y clases organizadas.

Además, se fortalecieron los conocimientos sobre el uso de archivos de texto para almacenar datos de manera permanente, mejorando la organización y reutilización del código. Finalmente, los ejercicios ayudaron a comprender mejor el manejo de información dentro del aplicativo web del proyecto final, facilitando el almacenamiento y control de datos en el sistema.

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones



- Se logró desarrollar programas en C + + y Java utilizando clases, objetos y métodos, lo que permitió organizar mejor el código y disminuir errores durante el desarrollo de los ejercicios.
- La implementación de archivos de texto permite guardar y recuperar información correctamente, facilitando el manejo permanente de los datos dentro de los sistemas desarrollados.
- El uso de programación orientada a objetos ayudó a reutilizar métodos y mejorar la estructura de los programas, haciendo más sencillo el mantenimiento y comprensión del código.
- Los conocimientos adquiridos en la guía APE-05 contribuyeron al desarrollo del aplicativo web del proyecto final, mejorando el manejo, almacenamiento y organización de la información dentro del sistema.

2.10 Recomendaciones

- Utilizar nombres claros para variables, clases y métodos para facilitar la comprensión del código.
- Validar correctamente los datos ingresados para evitar errores en el manejo de archivos y operaciones del sistema.
- Mantener organizado y comentado el código para facilitar futuras modificaciones y mejoras.
- Realizar pruebas constantes de lectura y escritura de archivos para comprobar el correcto funcionamiento de los programas.

2.11 Anexos

2.11.1 Ejercicio 1

2.11.1.1 Enunciado

Registro de Estudiantes

Crear un programa que permita registrar estudiantes en un archivo de texto.

Requerimientos:

- Solicitar cédula, nombre, carrera y promedio.
- Guardar la información en estudiantes.txt.
- Mostrar mensaje de confirmación.

Aplicar:

- Clases
- Objetos
- Métodos
- Escritura de archivos

2.11.1.2 Análisis del Problema



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



El problema requiere el diseño e implementación de un sistema básico de persistencia de datos en archivos de texto (.txt) utilizando el paradigma orientado a objetos.

Entradas: Datos del estudiante ingresados por teclado:

1. Cédula (Identificador único, comúnmente manejado como cadena de texto para preservar ceros a la izquierda).
2. Nombre Completo.
3. Carrera Universitaria.
4. Promedio Académico (Valor numérico decimal).

Procesos:

1. Instanciar un objeto de la clase Estudiante con los datos capturados.
2. Formatear los atributos del objeto en una línea de texto (por ejemplo, separados por comas o tabulaciones).
3. Abrir el archivo estudiantes.txt en modo de escritura/adición (*append*) para no borrar registros previos.
4. Escribir los datos y cerrar el flujo del archivo adecuadamente para asegurar la persistencia.

Salidas: Un mensaje en la consola que confirme al usuario que el registro se guardó con éxito y el archivo físico actualizado en el disco.

2.11.1.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
cedula	String	String	Almacena el documento de identidad (se usa texto para permitir ceros a la izquierda).
nombre	String	String	Almacena los nombres y apellidos completos del estudiante.
carrera	String	String	Almacena la carrera universitaria que cursa.
promedio	double	double	Almacena la nota promedio en formato decimal de doble precisión.



teclado	Scanner	Scanner	Objeto que permite capturar los datos que el usuario escribe en la consola.
cedula	String	String	Variable temporal que recibe el texto de la cédula por teclado.
nombre	String	String	Variable temporal que recibe el nombre completo por teclado.
carrera	String	String	Arreglo de objetos donde se Variable temporal que recibe la carrera por teclado. los productos registrados.
promedio	double	double	Variable temporal que recibe la nota por teclado.

2.11.1.4 Algoritmo Paso a Paso

Inicio

Definir la estructura de la clase Estudiante con sus atributos privados (cedula, nombre, carrera, promedio), su constructor y sus métodos de acceso (*getters*) o el método para empaquetar los datos.

Solicitar al usuario que ingrese los datos del estudiante por teclado: Cédula, Nombre, Carrera y Promedio.

Crear (instanciar) un objeto de tipo Estudiante pasando los datos recolectados al constructor.

Intentar abrir el archivo estudiantes.txt en modo de adición para guardar la información al final del archivo.

Si el archivo se abrió correctamente:

- Obtener la información formateada del objeto Estudiante.
- Escribir dicha información en una nueva línea del archivo.
- Cerrar el archivo de texto.
- Mostrar en pantalla el mensaje de confirmación: *"Estudiante registrado con éxito."*

De lo contrario (Si falla la apertura):

Mostrar un mensaje de error indicando que no se pudo acceder al archivo.

Fin



2.11.1.5 Pseudocódigo

```
// =====  
// Subproceso para simular el comportamiento de guardar  
// =====  
SubProceso registrarEnArchivo(cedula, nombre, carrera, promedio)  
    Definir linea Como Texto  
  
    // Formatear los datos del estudiante en una sola línea  
    linea <- cedula + " | " + nombre + " | " + carrera + " | " + ConvertirATexto(promedio)  
  
    // Intentar abrir y escribir en el archivo estudiantes.txt  
    // Nota: PseInt añade la línea al final si el archivo ya existe en configuraciones  
    estándar  
    Escribir En Archivo "estudiantes.txt" linea  
  
    // Mensaje de confirmación requerido  
    Escribir ""  
    Escribir "[CONFIRMACIÓN]: Estudiante registrado con éxito en estudiantes.txt."  
FinSubProceso  
  
// =====  
// Algoritmo Principal  
// =====  
Algoritmo Registro_Estudiantes  
    // Declaración de Variables  
    Definir cedula, nombre, carrera Como Texto  
    Definir promedio Como Real  
  
    Escribir "=== REGISTRO DE ESTUDIANTES ==="  
  
    // Solicitar datos por teclado  
    Escribir "Ingrese el número de cédula:"  
    Leer cedula  
  
    Escribir "Ingrese el nombre completo del estudiante:"  
    Leer nombre
```



Escribir "Ingrese la carrera universitaria:"

Leer carrera

Escribir "Ingrese el promedio académico:"

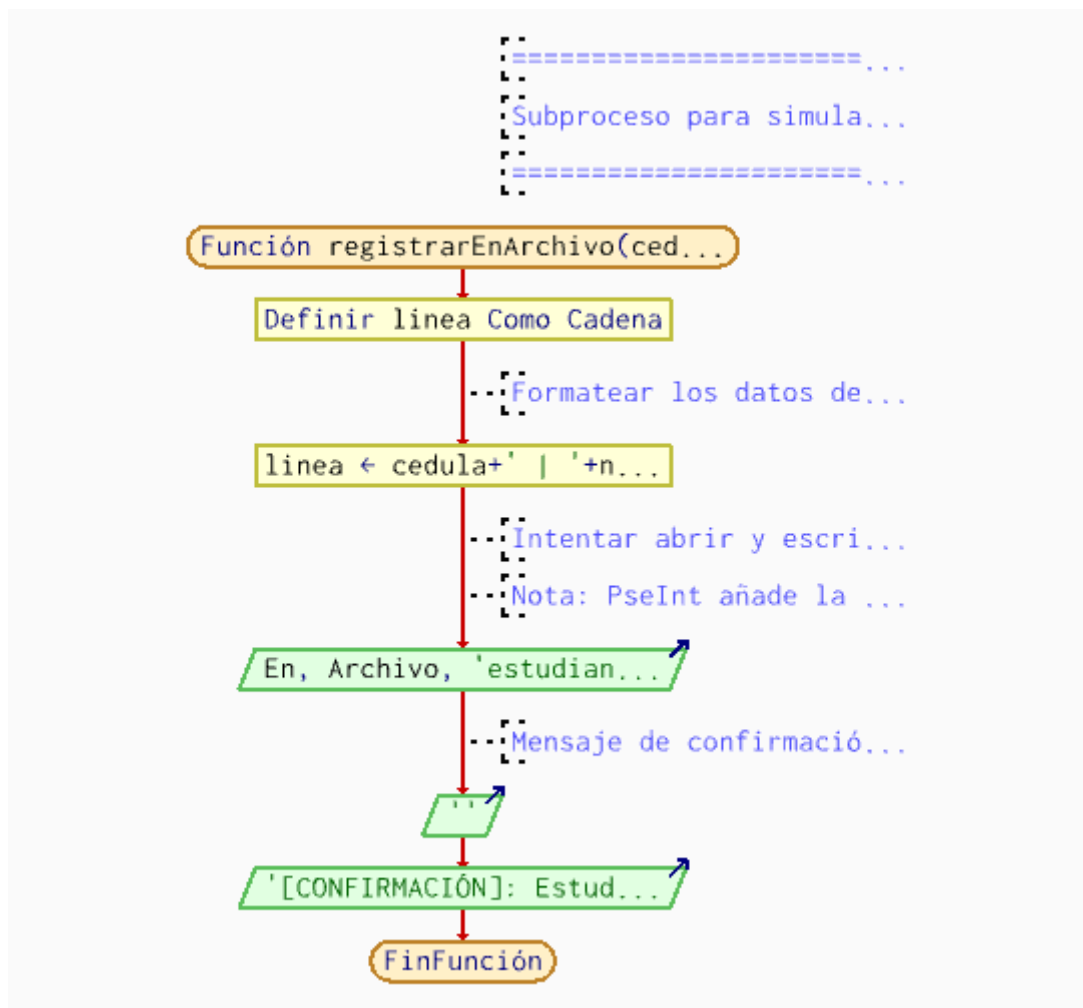
Leer promedio

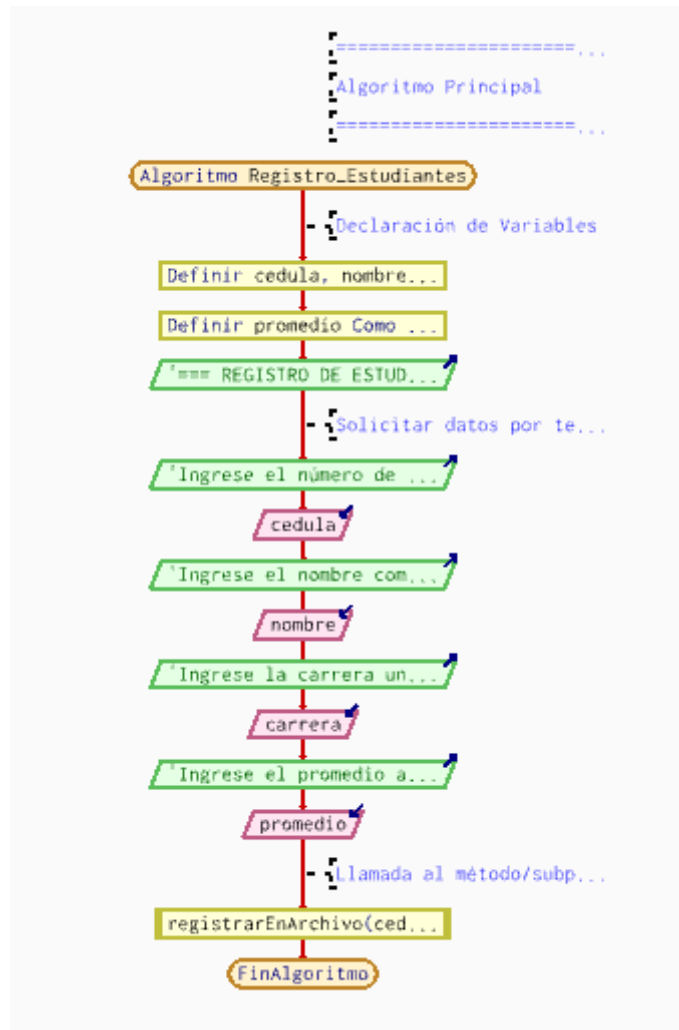
// Llamada al método/subproceso de escritura

registrarEnArchivo(cedula, nombre, carrera, promedio)

FinAlgoritmo

2.11.1.6 Diagrama de flujo





2.11.1.7 Codificación C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
// =====
```

```
// DEFINICIÓN DE LA CLASE ESTUDIANTE
```

```
// =====
```

```
class Estudiante {
```

```
private:
```

```
    string cedula;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
string nombre;  
string carrera;  
double promedio;
```

public:

```
// Constructor para inicializar el objeto
```

```
Estudiante(string _cedula, string _nombre, string _carrera, double _promedio) {  
    cedula = _cedula;  
    nombre = _nombre;  
    carrera = _carrera;  
    promedio = _promedio;  
}
```

```
// Métodos Getters (Encapsulamiento)
```

```
string getCedula() { return cedula; }  
string getNombre() { return nombre; }  
string getCarrera() { return carrera; }  
double getPromedio() { return promedio; }  
};
```

```
// =====
```

```
// PROGRAMA PRINCIPAL
```

```
// =====
```

```
int main() {
```

```
    // Declaración de variables locales para la captura
```

```
    string ced, nom, carr;
```

```
    double prom;
```

```
    cout << "=== REGISTRO DE ESTUDIANTES ===" << endl;
```

```
    // Solicitar datos al usuario
```

```
    cout << "Ingrese la cedula: ";
```

```
    cin >> ced;
```

```
    cin.ignore(); // Limpia el buffer de entrada para evitar problemas con getline()
```

```
    cout << "Ingrese el nombre completo: ";
```

```
    getline(cin, nom);
```



```
cout << "Ingrese la carrera: ";
getline(cin, carr);

cout << "Ingrese el promedio: ";
cin >> prom;

// Instancia del Objeto (Creación del objeto estudiante con los datos leídos)
Estudiante estudiante(ced, nom, carr, prom);

// Escritura en archivo usando la librería fstream
// ios::app (append) asegura que los nuevos datos se agreguen al final sin borrar lo
// existente
ofstream archivoOut;
archivoOut.open("estudiantes.txt", ios::app);

// Validación y escritura
if (archivoOut.is_open()) {
    // Almacenamos los datos separados por comas (formato CSV básico)
    archivoOut << estudiante.getCedula() << ","
        << estudiante.getNombre() << ","
        << estudiante.getCarrera() << ","
        << estudiante.getPromedio() << "\n";

    // Cierre del flujo del archivo
    archivoOut.close();

    // Mensaje de confirmación requerido
    cout << "\n[CONFIRMACION]: Estudiante registrado con exito en
estudiantes.txt." << endl;
} else {
    cout << "\n[ERROR]: No se pudo abrir o crear el archivo estudiantes.txt." << endl;
}

return 0;
}
```

2.11.1.8 Codificación Java



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

// =====
// DEFINICIÓN DE LA CLASE ESTUDIANTE
// =====

class Estudiante {
    // Atributos privados (Encapsulamiento)
    private String cedula;
    private String nombre;
    private String carrera;
    private double promedio;

    // Constructor para inicializar el objeto
    public Estudiante(String cedula, String nombre, String carrera, double promedio) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.promedio = promedio;
    }

    // Métodos Getters de acceso
    public String getCedula() { return cedula; }
    public String getNombre() { return nombre; }
    public String getCarrera() { return carrera; }
    public double getPromedio() { return promedio; }
}

// =====
// CLASE PRINCIPAL (Debe estar en un archivo llamado Main.java)
// =====

public class Main {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("== REGISTRO DE ESTUDIANTES ==");

// Solicitar datos al usuario
System.out.print("Ingrese la cédula: ");
String cedula = teclado.nextLine();

System.out.print("Ingrese el nombre completo: ");
String nombre = teclado.nextLine();

System.out.print("Ingrese la carrera: ");
String carrera = teclado.nextLine();

System.out.print("Ingrese el promedio: ");
double promedio = teclado.nextDouble();

// Instancia del Objeto Estudiante
Estudiante estudiante = new Estudiante(cedula, nombre, carrera, promedio);

// Escritura de archivos limpia de Inlay Hints (parámetros fantasma del editor)
try (FileWriter fw = new FileWriter("estudiantes.txt", true);
    PrintWriter pw = new PrintWriter(fw)) {

    // Escribir los datos del objeto formateados por comas
    pw.println(estudiante.getCedula() + "," +
        estudiante.getNombre() + "," +
        estudiante.getCarrera() + "," +
        estudiante.getPromedio());

    // Mensaje de confirmación requerido
    System.out.println("\n[CONFIRMACIÓN]: Estudiante registrado con éxito en
estudiantes.txt.");

} catch (IOException e) {
    System.out.println("\n[ERROR]: No se pudo escribir en el archivo: " +
e.getMessage());
}
```



```
teclado.close();
```

```
}
```

```
}
```

2.11.1.9 Prueba de Escritorio

2.11.2 Ejercicio 2

2.11.2.1 Enunciado

Lectura de Archivo de Estudiantes

Leer el archivo generado en el ejercicio anterior.

Requerimientos:

- Mostrar todos los registros almacenados.
- Contar cuántos estudiantes existen.
- Mostrar el promedio general.

Aplicar:

- Lectura de archivos
- Ciclos
- Arreglos
- Métodos

2.11.2.2 Análisis del Problema

El problema consiste en recuperar información previamente persistida en un archivo plano (estudiantes.txt), estructurarla temporalmente en memoria, procesarla secuencialmente y generar un reporte estadístico por pantalla.

Entradas: El archivo de texto estudiantes.txt donde cada línea representa un estudiante con sus campos separados por comas (formato CSV básico): cedula,nombre,carrera,promedio.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Procesos:

- Abrir el archivo en modo de lectura.
- Leer línea por línea el archivo utilizando un ciclo.
- Por cada línea, separar la cadena por sus delimitadores (comas) para extraer los atributos individuales.
- Instanciar objetos de la clase Estudiante e introducirlos dentro de un arreglo (estructura de datos indexada).
- Recorrer el arreglo para mostrar cada registro por consola.
- Contar la cantidad total de elementos dentro del arreglo.
- Sumar los promedios individuales acumulados y dividirlos para el total con el fin de obtener el promedio general.

Salidas: La lista detallada de estudiantes en pantalla.

- La cantidad total de estudiantes registrados.
- El promedio académico general de toda la institución.

2.11.2.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
listaEstudiantes	Estudiante[]	Estudiante[]	Arreglo de objetos con capacidad fija (100) para almacenar los registros en memoria dinámica.
contador	int	int	Variable entera que cuenta los estudiantes reales procesados y sirve como índice del arreglo.
fr	FileReader	FileReader	Flujo de caracteres que abre la conexión directa con estudiantes.txt.
br	BufferedReader	BufferedReader	Filtro que optimiza el rendimiento y lee el archivo línea por línea usando <code>.readLine()</code> .



linea	String	String	Variable temporal que almacena el texto completo de cada fila del archivo.
sumaPromedios	double	String	Acumulador matemático que suma las notas de todos los alumnos en el arreglo.
i	int	int	Variable temporal que recibe el nombre completo por teclado.
promedioGeneral	double	double	Guarda el resultado de la división entre sumaPromedios y contador.
e	IOException	IOException	Objeto que captura y gestiona fallos físicos de lectura si el archivo no existe.

2.11.2.4 Algoritmo Paso a Paso

Inicio

Declarar un arreglo capaz de almacenar objetos de tipo Estudiante y un contador inicializado en cero.

Intentar abrir el archivo estudiantes.txt en modo lectura.

Si el archivo no existe o no se puede abrir, mostrar un mensaje de error y terminar.

Mientras el archivo tenga líneas por leer:

- Leer una línea de texto completa.
- Separar (parsear) la línea usando la coma , como separador para obtener: cédula, nombre, carrera y promedio.
- Convertir la porción del promedio de texto a tipo decimal.
- Crear un objeto Estudiante con esos datos.
- Guardar el objeto en el arreglo en la posición apuntada por contador.
- Incrementar contador en 1.

Cerrar el archivo de texto.

Inicializar sumaPromedios en cero.

Para cada índice i desde 0 hasta contador - 1 (recorrido del arreglo):

- Imprimir en pantalla los atributos del estudiante en la posición i.
- Sumar el promedio del estudiante actual a sumaPromedios.



Si contador es mayor que cero:

- $\text{Calcular promedioGeneral} = \text{sumaPromedios} / \text{contador}$.

De lo contrario:

- $\text{promedioGeneral} = 0$.

Mostrar en pantalla el total de estudiantes (contador) y el promedioGeneral.

Fin

2.11.2.5 Pseudocódigo

Algoritmo LecturaEstudiantes

// Definición de arreglos paralelos con dimensiones fijas

Dimension listaCedulas[100]

Dimension listaNombres[100]

Dimension listaCarreras[100]

Dimension listaPromedios[100]

Definir listaCedulas, listaNombres, listaCarreras Como Texto

Definir listaPromedios Como Real

Definir contador, i Como Entero

Definir sumaPromedios, promedioGeneral Como Real

Definir continuar Como Texto

contador <- 0

sumaPromedios <- 0

continuar <- "S"

Escribir "=== CARGA DE REGISTROS SIMULADA DESDE FUENTE DE
DATOS ==="

Escribir ""

// Ciclo para simular la lectura/ingreso de la base de datos de estudiantes

Mientras (continuar = "S" O continuar = "s") Y contador < 100 Hacer

 contador <- contador + 1

 Escribir "Leyendo registro #", contador

 Escribir "Ingrese Cédula:"

 Leer listaCedulas[contador]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Ingrese Nombre Completo:"

Leer listaNombres[contador]

Escribir "Ingrese Carrera:"

Leer listaCarreras[contador]

Escribir "Ingrese Promedio Académico:"

Leer listaPromedios[contador]

Escribir "¿Desea procesar otro registro del archivo? (S/N):"

Leer continuar

Escribir "-----"

FinMientras

// --- DESPLIEGUE DE REPORTE ---

Escribir ""

Escribir "=====

Escribir "=== REPORTE DE ESTUDIANTES PROCESADOS ==="

Escribir "=====

// Ciclo estructurado para recorrer los arreglos y calcular métricas

Para i <- 1 Hasta contador Con Paso 1 Hacer

Escribir "Estudiante #", i

Escribir " -> Cédula: ", listaCedulas[i]

Escribir " -> Nombre: ", listaNombres[i]

Escribir " -> Carrera: ", listaCarreras[i]

Escribir " -> Promedio: ", listaPromedios[i]

Escribir "-----"

// Acumulador para la media aritmética

sumaPromedios <- sumaPromedios + listaPromedios[i]

FinPara

// --- CÁLCULO DE MÉTRICAS REQUERIDAS ---

Si contador > 0 Entonces

promedioGeneral <- sumaPromedios / contador



Sino

promedioGeneral <- 0

FinSi

Escribir ""

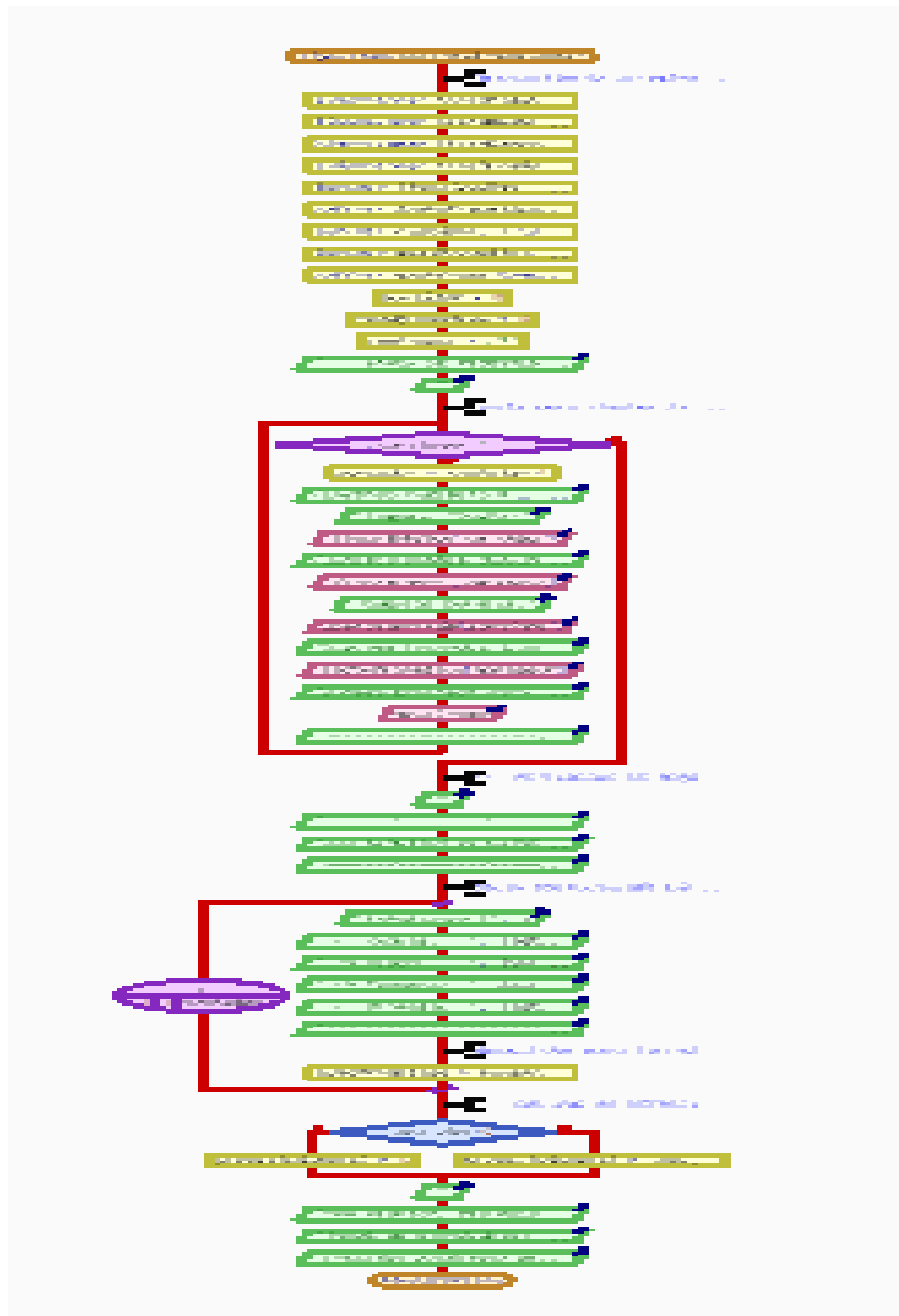
Escribir ">>> METRICAS DEL REQUERIMIENTO <<<"

Escribir "Total de estudiantes existentes: ", contador

Escribir "Promedio academico general: ", promedioGeneral

FinAlgoritmo

2.11.2.6 Diagrama de flujo



2.11.2.7 Codificación C++

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
```

```
using namespace std;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// =====  
// DEFINICIÓN DE LA CLASE ESTUDIANTE  
// =====  
class Estudiante {  
private:  
    string cedula;  
    string nombre;  
    string carrera;  
    double promedio;  
  
public:  
    // Constructor por defecto (necesario para inicializar arreglos en C++)  
    Estudiante() {  
        cedula = "";  
        nombre = "";  
        carrera = "";  
        promedio = 0.0;  
    }  
  
    // Constructor parametrizado  
    Estudiante(string _cedula, string _nombre, string _carrera, double _promedio) {  
        cedula = _cedula;  
        nombre = _nombre;  
        carrera = _carrera;  
        promedio = _promedio;  
    }  
  
    // Métodos Getters para acceder a los atributos encapsulados  
    string getCedula() { return cedula; }  
    string getNombre() { return nombre; }  
    string getCarrera() { return carrera; }  
    double getPromedio() { return promedio; }  
};  
  
// =====  
// PROGRAMA PRINCIPAL  
// =====
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
int main() {  
    // Requerimiento: Uso de Arreglos (Capacidad para 100 estudiantes)  
    Estudiante listaEstudiantes[100];  
    int contador = 0; // Cuenta los estudiantes existentes  
  
    // Requerimiento: Lectura de archivos  
    ifstream archivoIn("estudiantes.txt");  
  
    if (!archivoIn.is_open()) {  
        cout << "[ERROR]: No se pudo abrir el archivo estudiantes.txt o el archivo no  
existe." << endl;  
        return 1;  
    }  
  
    string linea;  
    // Requerimiento: Ciclo para leer el archivo línea por línea  
    while (getline(archivoIn, linea)) {  
        if (linea.empty()) continue; // Ignorar líneas vacías de seguridad  
  
        stringstream ss(linea);  
        string ced, nom, carr, tempProm;  
        double prom;  
  
        // Separar la línea usando la coma como delimitador (Tokenización)  
        getline(ss, ced, ',');  
        getline(ss, nom, ',');  
        getline(ss, carr, ',');  
        getline(ss, tempProm, ',');  
  
        // Convertir el promedio que viene como texto a tipo double  
        prom = stod(tempProm);  
  
        // Guardar el objeto instanciado dentro del arreglo  
        listaEstudiantes[contador] = Estudiante(ced, nom, carr, prom);  
        contador++;  
    }  
}
```



```
// Cerrar el archivo después de cargar los datos en memoria
archivoIn.close();

// =====
// MOSTRAR RESULTADOS Y CÁLCULOS
// =====

cout << "=== MOSTRANDO TODOS LOS REGISTROS ALMACENADOS ==="
<< endl;

double sumaPromedios = 0;

// Ciclo para recorrer el arreglo de objetos
for (int i = 0; i < contador; i++) {
    cout << "Estudiante #" << (i + 1) << endl;
    cout << " -> Cedula: " << listaEstudiantes[i].getCedula() << endl;
    cout << " -> Nombre: " << listaEstudiantes[i].getNombre() << endl;
    cout << " -> Carrera: " << listaEstudiantes[i].getCarrera() << endl;
    cout << " -> Promedio: " << listaEstudiantes[i].getPromedio() << endl;
    cout << "-----" << endl;

    // Acumular los promedios para la estadística general
    sumaPromedios += listaEstudiantes[i].getPromedio();
}

// Calcular el promedio general controlando la división para cero
double promedioGeneral = (contador > 0) ? (sumaPromedios / contador) : 0.0;

// Mostrar requerimientos de salida finales
cout << "\n>>> REPORTE METRICO FINAL <<<" << endl;
cout << "Total de estudiantes existentes: " << contador << endl;
cout << "Promedio general del grupo: " << promedioGeneral << endl;

return 0;
}
```

2.11.2.8 Codificación Java

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// =====  
// DEFINICIÓN DE LA CLASE ESTUDIANTE  
// =====  
  
class Estudiante {  
    private String cedula;  
    private String nombre;  
    private String carrera;  
    private double promedio;  
  
    // Constructor parametrizado  
    public Estudiante(String cedula, String nombre, String carrera, double promedio) {  
        this.cedula = cedula;  
        this.nombre = nombre;  
        this.carrera = carrera;  
        this.promedio = promedio;  
    }  
  
    // Métodos Getters (Encapsulamiento)  
    public String getCedula() { return cedula; }  
    public String getNombre() { return nombre; }  
    public String getCarrera() { return carrera; }  
    public double getPromedio() { return promedio; }  
}  
  
// =====  
// CLASE PRINCIPAL (Debe llamarse Main.java)  
// =====  
  
public class Main {  
    public static void main(String[] args) {  
        // Requerimiento: Uso de Arreglos (Inicialización de arreglo estático con tope de  
100)  
        Estudiante[] listaEstudiantes = new Estudiante[100];  
        int contador = 0; // Cuenta cuántos estudiantes existen en el archivo  
  
        // Requerimiento: Lectura de archivos mediante bloques seguros  
        try (FileReader fr = new FileReader("estudiantes.txt"));
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
BufferedReader br = new BufferedReader(fr)) {

String linea;
// Requerimiento: Ciclo para leer el archivo hasta que no haya más registros
while ((linea = br.readLine()) != null) {
    // Evitar procesar líneas en blanco accidentales
    if (linea.trim().isEmpty()) continue;

    // Separar la cadena por comas
    String[] campos = linea.split(",");

    String ced = campos[0];
    String nom = campos[1];
    String carr = campos[2];
    double prom = Double.parseDouble(campos[3]); // Conversión de Texto a
Double

    // Almacenar el objeto instanciado dentro de la posición actual del arreglo
    listaEstudiantes[contador] = new Estudiante(ced, nom, carr, prom);
    contador++;
}

} catch (IOException e) {
    System.out.println("[ERROR]: No se pudo leer el archivo o aún no ha sido
creado: " + e.getMessage());
    return;
}

// =====
// PROCESAMIENTO DE DATOS Y MÉTRICAS
// =====

    System.out.println("==== MOSTRANDO TODOS LOS REGISTROS
ALMACENADOS ===");
    double sumaPromedios = 0;

    // Ciclo para recorrer el arreglo de objetos estructurado
    for (int i = 0; i < contador; i++) {
```



```
System.out.println("Estudiante #" + (i + 1));
System.out.println(" -> Cédula: " + listaEstudiantes[i].getCedula());
System.out.println(" -> Nombre: " + listaEstudiantes[i].getNombre());
System.out.println(" -> Carrera: " + listaEstudiantes[i].getCarrera());
System.out.println(" -> Promedio: " + listaEstudiantes[i].getPromedio());
System.out.println("-----");

// Acumular la suma de notas para el cálculo final
sumaPromedios += listaEstudiantes[i].getPromedio();
}

// Requerimiento: Mostrar promedio general controlando división para cero
double promedioGeneral = (contador > 0) ? (sumaPromedios / contador) : 0.0;

// Salidas en pantalla solicitadas por el enunciado
System.out.println("\n>>> REPORTE ESTADÍSTICO FINAL <<<");
System.out.println("Total de estudiantes existentes: " + contador);
System.out.println("Promedio académico general: " + promedioGeneral);
}
}
```

2.11.2.9 Prueba de Escritorio

```
"C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\apeee.exe"
=== MOSTRANDO TODOS LOS REGISTROS ALMACENADOS ===
Estudiante #1
-> Cedula: 0850981929
-> Nombre: Samuel Bravo
-> Carrera: Software
-> Promedio: 9
-----
>>> REPORTE METRICO FINAL <<<
Total de estudiantes existentes: 1
Promedio general del grupo: 9
Process returned 0 (0x0) execution time : 2.717 s
Press any key to continue.
```

2.11.3 Ejercicio 3

2.11.3.1 Enunciado

Sistema de Inventario Básico

Crear un sistema para registrar productos.

Requerimientos:



- Guardar código, nombre, cantidad y precio.
- Guardar datos en inventario.txt.
- Calcular el valor total del inventario.

Aplicar:

- POO
- Archivos
- Métodos

2.11.3.2 Análisis del Problema

El problema consiste en desarrollar un sistema básico de inventario utilizando programación orientada a objetos en C++ y Java. El programa debe permitir registrar productos ingresando datos como código, nombre, cantidad y precio. Además, la información debe guardarse en un archivo de texto llamado inventario.txt para mantener un registro de los productos almacenados.

El sistema también debe calcular automáticamente el valor total del inventario multiplicando la cantidad de productos por su precio. Para realizar este ejercicio se aplican clases, objetos, métodos y manejo de archivos, permitiendo organizar mejor el código y facilitar el almacenamiento de la información.

2.11.3.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
codigo	int	int	Almacena el código del producto.
nombre	string	String	Guarda el nombre del producto.
cantidad	int	int	Almacena la cantidad disponible del producto.
precio	double	double	Guarda el precio unitario del producto.
totalInventario	double	double	Almacena el valor total del inventario.



cantidadProductos	int	int	Guarda la cantidad de productos que el usuario desea registrar.
i	int	int	Variable utilizada para controlar los ciclos repetitivos.
productos	Producto[]	Producto[]	Arreglo de objetos donde se almacenan los productos registrados.
archivo	ofstream	FileWriter	Permite guardar la información en el archivo inventario.txt.

2.11.3.4 Algoritmo Paso a Paso

Inicio

Definir una clase llamada Producto con atributos para código, nombre, cantidad y precio

Definir variables para almacenar la cantidad de productos y el valor total del inventario

Solicitar al usuario la cantidad de productos que desea registrar

Crear un arreglo de objetos de tipo Producto según la cantidad ingresada

Crear y abrir el archivo inventario.txt para guardar la información

Recorrer el arreglo utilizando un ciclo desde la primera posición hasta la última posición ingresada por el usuario

Dentro del ciclo solicitar el código, nombre, cantidad y precio de cada producto

Guardar la información ingresada en el objeto correspondiente

Calcular el valor total de cada producto multiplicando la cantidad por el precio

Sumar el valor de cada producto al total general del inventario

Guardar la información de cada producto en el archivo inventario.txt

Mostrar en pantalla los datos registrados de cada producto

Mostrar el valor total del inventario



Cerrar el archivo para guardar correctamente la información

Fin

2.11.3.5 Pseudocódigo

Algoritmo SistemaInventario

// Declaración de variables

Definir i, codigo, cantidad, cantidadProductos Como Entero

Definir nombre Como Cadena

Definir precio, totalProducto, totalInventario Como Real

// Inicializar el total general del inventario

totalInventario <- 0

// Solicitar la cantidad de productos

Escribir "Ingrese la cantidad de productos: "

Leer cantidadProductos

// Ciclo para registrar productos

Para i <- 1 Hasta cantidadProductos Hacer

// Ingreso de datos del producto

Escribir "Ingrese el codigo del producto: "

Leer codigo

Escribir "Ingrese el nombre del producto: "

Leer nombre

Escribir "Ingrese la cantidad del producto: "

Leer cantidad

Escribir "Ingrese el precio del producto: "

Leer precio

// Calcular el total del producto



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



totalProducto <- cantidad * precio

// Sumar al total general

totalInventario <- totalInventario + totalProducto

// Mostrar información del producto

Escribir "-----"

Escribir "Codigo: ", codigo

Escribir "Nombre: ", nombre

Escribir "Cantidad: ", cantidad

Escribir "Precio: \$", precio

Escribir "Total del producto: \$", totalProducto

Escribir "-----"

FinPara

// Mostrar total general del inventario

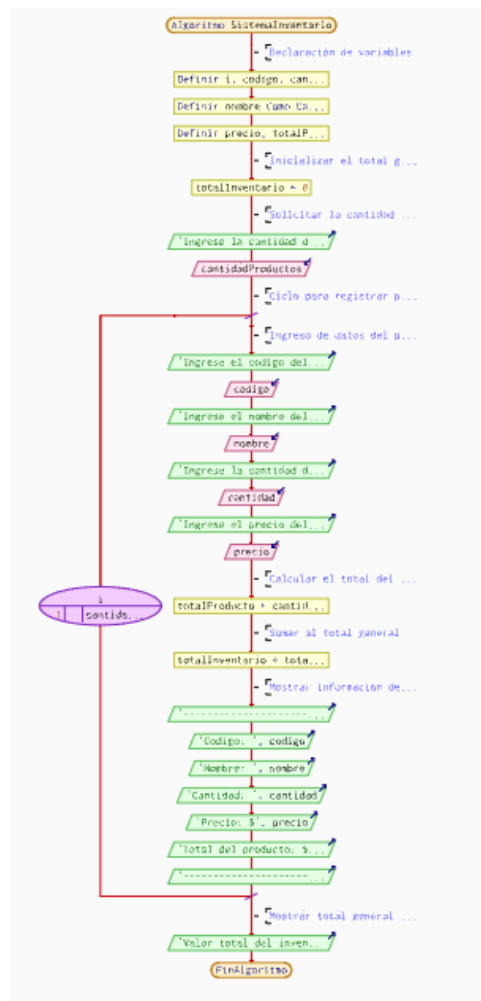
Escribir "Valor total del inventario: \$", totalInventario

FinAlgoritmo

2.11.3.6 Diagrama de flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



2.11.3.7 Codificación C++

2.11.3.7.1 Clase Producto.h

// Librerías necesarias

```
#include <iostream>
```

```
using namespace std;
```

// Creación de la clase Producto

```
class Producto {
```

// Atributos privados

```
private:
```

```
    int codigo;
```

```
    string nombre;
```

```
    int cantidad;
```

```
    double precio;
```



public:

```
// Constructor vacío
Producto() {

    codigo = 0;
    nombre = "";
    cantidad = 0;
    precio = 0;
}

// Métodos SET para guardar datos
void setCodigo(int c) {
    codigo = c;
}

void setNombre(string n) {
    nombre = n;
}

void setCantidad(int cant) {
    cantidad = cant;
}

void setPrecio(double p) {
    precio = p;
}

// Métodos GET para obtener información
int getCodigo() {
    return codigo;
}

string getNombre() {
    return nombre;
}
```



```
int getCantidad() {  
    return cantidad;  
}
```

```
double getPrecio() {  
    return precio;  
}
```

```
// Método para calcular el total del producto
```

```
double calcularTotal() {  
  
    return cantidad * precio;  
}
```

```
// Método para mostrar información
```

```
void mostrarProducto() {  
  
    cout << "\nCodigo: " << codigo << endl;  
    cout << "Nombre: " << nombre << endl;  
    cout << "Cantidad: " << cantidad << endl;  
    cout << "Precio: $" << precio << endl;  
    cout << "Total: $" << calcularTotal() << endl;  
}  
};
```

2.11.3.7.2 Clase Main.cpp

```
// Librerías necesarias
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include "Producto.h"
```

```
using namespace std;
```

```
int main() {
```

```
    // Variable para la cantidad de productos
```

```
    int cantidadProductos;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Variables temporales
int codigo;
int cantidad;
string nombre;
double precio;

// Variable para el total general
double totalInventario = 0;

// Pedir cuántos productos desea registrar el usuario
cout << "Ingrese la cantidad de productos: ";
cin >> cantidadProductos;

// Crear arreglo de objetos
Producto productos[cantidadProductos];

// Crear archivo para guardar datos
ofstream archivo("inventario.txt");

// Ciclo para registrar productos
for(int i = 0; i < cantidadProductos; i++) {

    cout << "\n===== " << endl;
    cout << "REGISTRO DEL PRODUCTO " << i + 1 << endl;
    cout << "===== " << endl;

    // Ingreso de datos
    cout << "Ingrese el codigo: ";
    cin >> codigo;

    cin.ignore();

    cout << "Ingrese el nombre: ";
    getline(cin, nombre);

    cout << "Ingrese la cantidad: ";
    cin >> cantidad;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "Ingrese el precio: ";
cin >> precio;

// Guardar datos en el objeto
productos[i].setCodigo(codigo);
productos[i].setNombre(nombre);
productos[i].setCantidad(cantidad);
productos[i].setPrecio(precio);

// Calcular total general
totalInventario += productos[i].calcularTotal();

// Guardar información en el archivo
archivo << "Codigo: " << codigo << endl;
archivo << "Nombre: " << nombre << endl;
archivo << "Cantidad: " << cantidad << endl;
archivo << "Precio: $" << precio << endl;
archivo << "Total: $" << productos[i].calcularTotal() << endl;
archivo << "-----" << endl;
}

// Cerrar archivo
archivo.close();

// Mostrar productos registrados
cout << "\n===== INVENTARIO =====" << endl;

for(int i = 0; i < cantidadProductos; i++) {

    productos[i].mostrarProducto();
}

// Mostrar total general del inventario
cout << "\nValor total del inventario: $" << totalInventario << endl;

return 0;
```



}

2.11.3.8 Codificación Java

2.11.3.8.1 Clase Producto.java

// Clase Producto

```
public class Producto {
```

```
    // Atributos privados
```

```
    private int codigo;
```

```
    private String nombre;
```

```
    private int cantidad;
```

```
    private double precio;
```

```
    // Constructor vacío
```

```
    public Producto() {
```

```
        codigo = 0;
```

```
        nombre = "";
```

```
        cantidad = 0;
```

```
        precio = 0;
```

```
    }
```

```
    // Métodos SET
```

```
    public void setCodigo(int codigo) {
```

```
        this.codigo = codigo;
```

```
    }
```

```
    public void setNombre(String nombre) {
```

```
        this.nombre = nombre;
```

```
    }
```

```
    public void setCantidad(int cantidad) {
```

```
        this.cantidad = cantidad;
```

```
    }
```

```
    public void setPrecio(double precio) {
```

```
        this.precio = precio;
```

```
    }
```



```
// Métodos GET
public int getCodigo() {
    return codigo;
}

public String getNombre() {
    return nombre;
}

public int getCantidad() {
    return cantidad;
}

public double getPrecio() {
    return precio;
}

// Método para calcular el total
public double calcularTotal() {

    return cantidad * precio;
}

// Método para mostrar información
public void mostrarProducto() {

    System.out.println("\nCodigo: " + codigo);
    System.out.println("Nombre: " + nombre);
    System.out.println("Cantidad: " + cantidad);
    System.out.println("Precio: $" + precio);
    System.out.println("Total: $" + calcularTotal());
}
}
```

2.11.3.8.2 Clase Main.java

```
// Librerías necesarias
import java.util.Scanner;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
import java.io.FileWriter;
import java.io.IOException;

public class Main {

    public static void main(String[] args) {

        // Objeto Scanner para ingresar datos
        Scanner entrada = new Scanner(System.in);

        // Pedir cantidad de productos
        System.out.print("Ingrese la cantidad de productos: ");
        int cantidadProductos = entrada.nextInt();

        // Crear arreglo de objetos
        Producto productos[] = new Producto[cantidadProductos];

        // Variables temporales
        int codigo;
        int cantidad;
        String nombre;
        double precio;

        // Variable para guardar el total general
        double totalInventario = 0;

        try {

            // Crear archivo
            FileWriter archivo = new FileWriter("inventariojava.txt");

            // Ciclo para registrar productos
            for(int i = 0; i < cantidadProductos; i++) {

                // Crear objeto
                productos[i] = new Producto();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("\n=====");
System.out.println("REGISTRO DEL PRODUCTO " + (i + 1));
System.out.println("=====");

// Ingreso de datos
System.out.print("Ingrese el codigo: ");
codigo = entrada.nextInt();

entrada.nextLine();

System.out.print("Ingrese el nombre: ");
nombre = entrada.nextLine();

System.out.print("Ingrese la cantidad: ");
cantidad = entrada.nextInt();

System.out.print("Ingrese el precio: ");
precio = entrada.nextDouble();

// Guardar datos
productos[i].setCodigo(codigo);
productos[i].setNombre(nombre);
productos[i].setCantidad(cantidad);
productos[i].setPrecio(precio);

// Calcular total general
totalInventario += productos[i].calcularTotal();

// Guardar en archivo
archivo.write("Codigo: " + codigo + "\n");
archivo.write("Nombre: " + nombre + "\n");
archivo.write("Cantidad: " + cantidad + "\n");
archivo.write("Precio: $" + precio + "\n");
archivo.write("Total: $" + productos[i].calcularTotal() + "\n");
archivo.write("-----\n");
}
```



```
// Cerrar archivo
archivo.close();

} catch(IOException e) {

    System.out.println("Error al crear el archivo");
}

// Mostrar inventario
System.out.println("\n===== INVENTARIO =====");

for(int i = 0; i < cantidadProductos; i++) {

    productos[i].mostrarProducto();
}

// Mostrar total general
System.out.println("\nValor total del inventario: $" + totalInventario);

// Cerrar Scanner
entrada.close();
}
}
```

2.11.3.9 Prueba de Escritorio

2.11.3.9.1 En C++



```
Ingrese el precio: 2

===== INVENTARIO =====

Codigo: 30
Nombre: Cafe
Cantidad: 2
Precio: $5
Total: $10

Codigo: 31
Nombre: Pollo
Cantidad: 1
Precio: $10
Total: $10

Codigo: 32
Nombre: Galletas
Cantidad: 8
Precio: $2
Total: $16

Valor total del inventario: $36
PS C:\Users\Usuario\OneDrive\Ape05\Actividades\output\output>
```

2.11.3.9.2 En Java

```
===== INVENTARIO =====

Codigo: 12
Nombre: Azucar
Cantidad: 3
Precio: $5.0
Total: $15.0

Codigo: 13
Nombre: Cocoa
Cantidad: 1
Precio: $8.0
Total: $8.0

Valor total del inventario: $23.0
PS C:\Users\Usuario\OneDrive\Ape05>
```

2.11.4 Ejercicio 4

2.11.4.1 Enunciado

Agenda Telefónica

- Crear una agenda de contactos usando archivos.
- Requerimientos:
- Guardar nombre, teléfono y correo.
- Agregar y buscar contactos.
- Mostrar todos los contactos.

Aplicar:

- Archivos
- Búsqueda



- Métodos

2.11.4.2 Análisis del Problema

El problema consiste en desarrollar un programa que permita gestionar una agenda telefónica mediante el uso de archivos de texto. El sistema debe permitir al usuario ingresar nuevos contactos, almacenarlos en un archivo, buscarlos por nombre y visualizar todos los registros guardados.

Para ello se requiere:

- Uso de archivos para persistencia de datos.
- Implementación de métodos para organizar las funciones.
- Uso de búsqueda dentro del archivo.

El archivo funcionará como base de datos simple (agenda.txt).

2.11.4.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
nombre	String	String	Almacena el nombre del contacto.
teléfono	String	String	Guarda el número telefónico del contacto.
correo	String	String	Almacena el correo electrónico del contacto.
linea	String	String	Guarda temporalmente cada línea leída del archivo.
opcion	int	int	Permite seleccionar una opción del menú.
encontrado	bool	boolean	Indica si el contacto fue encontrado en la búsqueda.
archivo	String	String	Representa el nombre del archivo donde se almacenan los datos (agenda.txt).



2.11.4.4 Algoritmo Paso a Paso

Iniciar programa

Mostrar menú de opciones

Leer opción del usuario

Según la opción:

Opción 1: Agregar contacto

- Pedir nombre, teléfono y correo
- Guardar datos en agenda.txt

Opción 2: Mostrar contactos

- Abrir archivo
- Leer línea por línea
- Mostrar contenido

Opción 3: Buscar contacto

- Pedir nombre
- Leer archivo
- Comparar datos
- Mostrar si se encuentra

Opción 4: Salir

- Finalizar programa

Repetir hasta que el usuario salga

2.11.4.5 Pseudocódigo

Algoritmo AgendaTalefonica

Definir nombres, telefonos, correos Como Cadena

Dimension nombres[100]

Dimension telefonos[100]

Dimension correos[100]

Definir opcion, i, total Como Entero

Definir buscar Como Cadena

Definir encontrado Como Logico



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



total <- 0

Repetir

Escribir "===== AGENDA TELEFONICA ====="

Escribir "1. Agregar contacto"

Escribir "2. Mostrar contactos"

Escribir "3. Buscar contacto"

Escribir "4. Salir"

Leer opcion

Segun opcion Hacer

1:

Si total < 100 Entonces

total <- total + 1

Escribir "Ingrese nombre:"

Leer nombres[total]

Escribir "Ingrese telefono:"

Leer telefonos[total]

Escribir "Ingrese correo:"

Leer correos[total]

Escribir "Contacto guardado"

Sino

Escribir "Agenda llena"

FinSi

2:

Escribir "=== LISTA DE CONTACTOS ==="

Si total = 0 Entonces

Escribir "No hay contactos"



Sino

Para i <- 1 Hasta total Hacer

Escribir nombres[i], " - ", telefonos[i], " - ", correos[i]

FinPara

FinSi

3:

encontrado <- Falso

Escribir "Ingrese nombre a buscar:"

Leer buscar

Para i <- 1 Hasta total Hacer

Si nombres[i] = buscar Entonces

Escribir "Encontrado:"

Escribir nombres[i], " - ", telefonos[i], " - ", correos[i]

encontrado <- Verdadero

FinSi

FinPara

Si encontrado = Falso Entonces

Escribir "Contacto no encontrado"

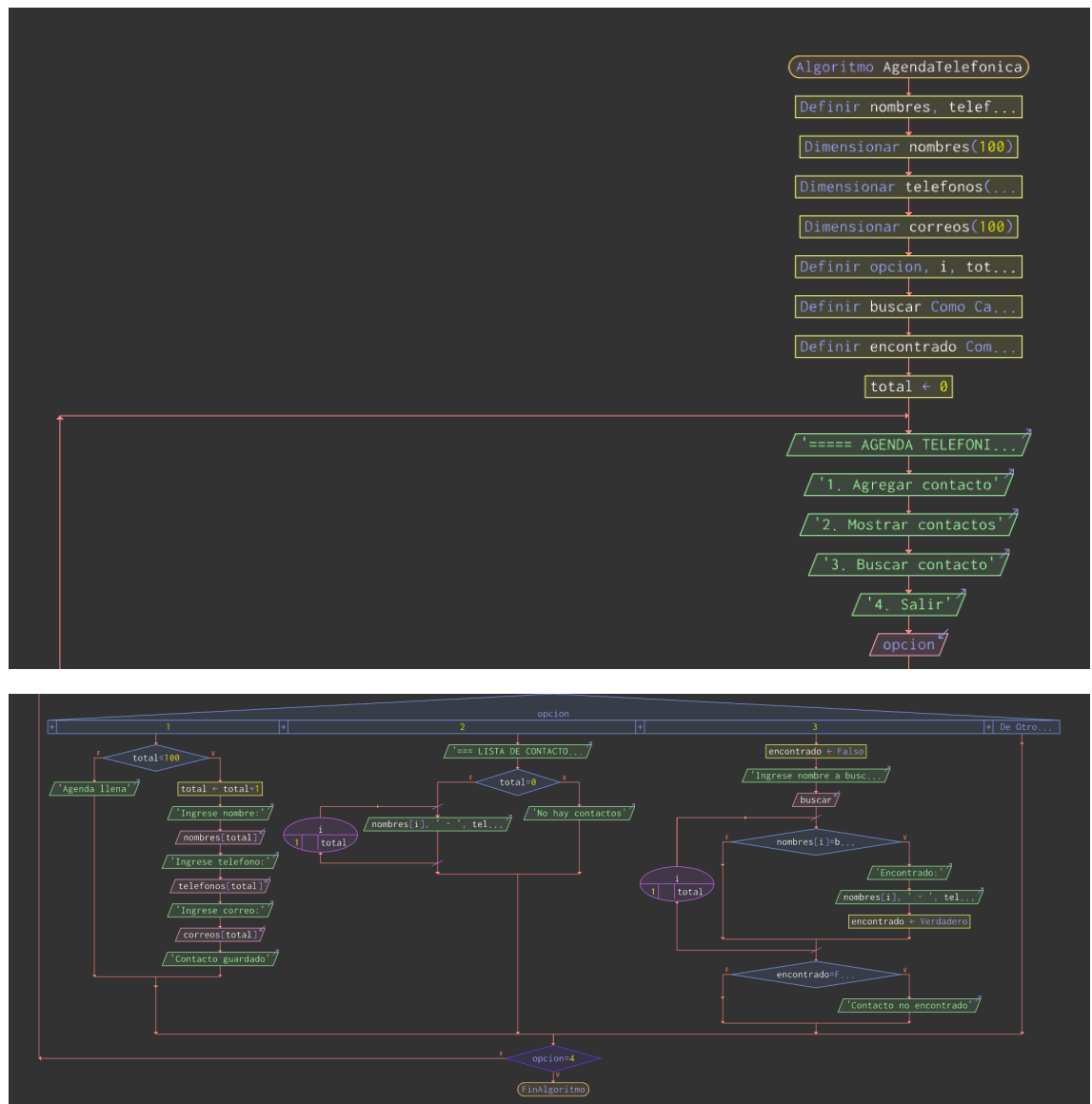
FinSi

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

2.11.4.6 Diagrama de flujo



2.11.4.7 Codificación C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Contacto {
```

```
public:
```

```
    string nombre, telefono, correo;
```

```
    Contacto(string n, string t, string c) {
```

```
        nombre = n;
```

```
        telefono = t;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        correo = c;
    }

    string toString() {
        return nombre + "," + telefono + "," + correo;
    }
};

string archivo = "agenda.txt";
void agregar() {
    ofstream file(archivo, ios::app);
    string n, t, c;

    cout << "Nombre: ";
    getline(cin, n);
    cout << "Telefono: ";
    getline(cin, t);
    cout << "Correo: ";
    getline(cin, c);

    Contacto con(n, t, c);
    file << con.toString() << endl;

    file.close();
    cout << "Contacto guardado\n";
}

void mostrar() {
    ifstream file(archivo);
    string linea;

    while (getline(file, linea)) {
        cout << linea << endl;
    }

    file.close();
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
void buscar() {
    ifstream file(archivo);
    string linea, nombre;
    bool encontrado = false;

    cout << "Nombre a buscar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) != string::npos) {
            cout << "Encontrado: " << linea << endl;
            encontrado = true;
        }
    }

    if (!encontrado)
        cout << "No encontrado\n";

    file.close();
}

int main() {
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Buscar\n4. Salir\n";
        cout << "Opcion: ";
        cin >> op;
        cin.ignore();

        switch (op) {
            case 1: agregar(); break;
            case 2: mostrar(); break;
            case 3: buscar(); break;
        }
    }
```



```
} while (op != 4);
```

```
return 0;
```

```
}
```

2.11.4.8 Codificación Java

```
// Librerías necesarias
```

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
// Clase Contacto
```

```
class Contacto {
```

```
    // Atributos
```

```
    String nombre;
```

```
    String telefono;
```

```
    String correo;
```

```
    // Constructor
```

```
    public Contacto(String n, String t, String c) {
```

```
        nombre = n;
```

```
        telefono = t;
```

```
        correo = c;
```

```
    }
```

```
    // Método para convertir los datos en texto
```

```
    public String toString() {
```

```
        return nombre + "," + telefono + "," + correo;
```

```
    }
```

```
}
```

```
// Clase principal
```

```
public class Main {
```

```
    // Nombre del archivo
```

```
    static String archivo = "agenda.txt";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Método para agregar contactos
public static void agregar(Scanner entrada) {

    try {

        // Abrir archivo en modo agregar
        FileWriter file = new FileWriter(archivo, true);

        String n, t, c;

        // Ingreso de datos
        System.out.print("Nombre: ");
        n = entrada.nextLine();

        System.out.print("Telefono: ");
        t = entrada.nextLine();

        System.out.print("Correo: ");
        c = entrada.nextLine();

        // Crear objeto contacto
        Contacto con = new Contacto(n, t, c);

        // Guardar en archivo
        file.write(con.toString() + "\n");

        // Cerrar archivo
        file.close();

        System.out.println("Contacto guardado");

    } catch(IOException e) {

        System.out.println("Error al guardar");
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Método para mostrar contactos
public static void mostrar() {

    try {

        // Abrir archivo para lectura
        BufferedReader file = new BufferedReader(new FileReader(archivo));

        String linea;

        // Leer línea por línea
        while((linea = file.readLine()) != null) {

            System.out.println(linea);
        }

        // Cerrar archivo
        file.close();

    } catch(IOException e) {

        System.out.println("Error al leer el archivo");
    }
}

// Método para buscar contacto
public static void buscar(Scanner entrada) {

    try {

        // Abrir archivo
        BufferedReader file = new BufferedReader(new FileReader(archivo));

        String linea;
        String nombre;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
boolean encontrado = false;

// Pedir nombre
System.out.print("Nombre a buscar: ");
nombre = entrada.nextLine();

// Buscar contacto
while((linea = file.readLine()) != null) {

    if(linea.contains(nombre)) {

        System.out.println("Encontrado: " + linea);
        encontrado = true;
    }
}

// Verificar si no existe
if(!encontrado) {

    System.out.println("No encontrado");
}

// Cerrar archivo
file.close();

} catch(IOException e) {

    System.out.println("Error al buscar");
}

}

// Método principal
public static void main(String[] args) {

    // Scanner para leer datos
    Scanner entrada = new Scanner(System.in);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
int op;

do {

    // Menú
    System.out.println("\n1. Agregar");
    System.out.println("2. Mostrar");
    System.out.println("3. Buscar");
    System.out.println("4. Salir");

    System.out.print("Opcion: ");
    op = entrada.nextInt();
    entrada.nextLine();

    // Opciones
    switch(op) {

        case 1:

            agregar(entrada);
            break;

        case 2:

            mostrar();
            break;

        case 3:

            buscar(entrada);
            break;

    }

} while(op != 4);

// Cerrar Scanner
entrada.close();
```



}

}

2.11.4.9 Prueba de Escritorio

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 1
Nombre: Michael
Telefono: 0979350748
Correo: msanchez3405@gmail.com
Contacto guardado
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 2
Michael,0979350478,mruiz3405@gmail.com
Michael,0979350748,msanchez3405@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 3
Nombre a buscar: Michael
Encontrado: Michael,0979350478,mruiz3405@gmail.com
Encontrado: Michael,0979350748,msanchez3405@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 4
PS C:\Users\HP\Documents\ALGORITMOS\Agenda> █
```

2.11.5 Ejercicio 5

2.11.5.1 Enunciado

Sistema de Notas

Registrar calificaciones de estudiantes.

Requerimientos:

- Guardar nombre y 3 notas.
- Calcular promedio.
- Determinar si fue aprobado o reprobado.



Aplicar:

- Métodos
- POO
- Archivos

2.11.5.2 Análisis del Problema

El propósito de este programa es crear un sistema de ingreso de notas en el cual se pueda ingresar datos de los estudiantes tales como su nombre y apellido, sus notas y su cédula de identidad por cada estudiante. También el ejercicio propuesto es una excelente práctica para poder mejorar las habilidades y el uso de POO con clases, objetos y métodos utilizados dentro del código; también se debe tomar en cuenta ciertas condiciones establecidas al momento de la codificación.

2.11.5.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
cedula	string	String	Variable para almacenar las cédulas de los estudiantes en arreglos posteriores.
nombre	string	String	Variable para almacenar los nombres de los estudiantes en arreglos posteriores.
apellido	string	String	Variable para almacenar los apellidos de los estudiantes en arreglos posteriores.
nota1	double	double	Variable para almacenar la nota 1 de los estudiantes en arreglos posteriores.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



nota2	double	double	Variable para almacenar la nota 2 de los estudiantes en arreglos posteriores.
nota3	double	double	Variable para almacenar la nota 3 de los estudiantes en arreglos posteriores.
promedio	double	double	Variable para almacenar el promedio de los estudiantes en arreglos posteriores.
estado	string	String	Variable para almacenar el estado académico de los estudiantes en arreglos posteriores.

2.11.5.4 Algoritmo Paso a Paso

Inicio

Definir la clase Estudiante.

Definir las variables cédula, nombre, apellido, nota1, nota2, nota3, promedio, estado.

Definir el constructor de forma pública con las variables establecidas.

Definir getters y setters para obtener y establecer las variables correspondientes con valores.

Definir una función para calcular el promedio.

Definir una función para determinar el estado de aprobado o reprobado.

Definir una función para poder imprimir la información de manera más eficiente.

Definir una función para verificar las notas que estén dentro del rango permitido.

Establecer la función principal o “main”.

Definir un vector estudiantes para un mejor manejo de información de las variables en general.

Establecer la cantidad de estudiantes.

Crear un ciclo “for” para solicitar y almacenar los datos.

Solicitar los datos al usuario. Solicitar las notas (dentro del rango).

Llamar al objeto estudiante.



Guardar toda la información almacenada en un vector (para una mejor impresión y manejo de los datos).

Definir contadores “aprobados” y “reprobados”.

Crear ciclo “for” para definir la cantidad de estudiantes aprobados y reprobados.

Imprimir en consola el reporte necesario de la información.

Fin

2.11.5.5 Pseudocódigo

Algoritmo Registro_Estudiantes_Algoritmos

// Definición de variables

Definir cantidadEstudiantes, i, aprobados, reprobados Como Entero

Definir cedula, nombre, apellido, estado Como Cadena

Definir nota1, nota2, nota3, promedio Como Real

// Arreglos para almacenar datos

Dimension cedulas[5]

Dimension nombres[5]

Dimension apellidos[5]

Dimension notas1[5]

Dimension notas2[5]

Dimension notas3[5]

Dimension promedios[5]

Dimension estados[5]

cantidadEstudiantes <- 5

aprobados <- 0

reprobados <- 0

Escribir "=== REGISTRO DE ESTUDIANTES - ALGORITMOS ==="

// Registro de estudiantes

Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer

Escribir ""

Escribir "--- Datos del Estudiante ", i + 1, " ---"

Escribir "Ingrese Cedula:"



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Leer cedula

Escribir "Ingrese Nombre:"

Leer nombre

Escribir "Ingrese Apellido:"

Leer apellido

// Validación Nota 1

Repetir

Escribir "Ingrese la Nota 1 (0 - 10):"

Leer nota1

Si $\text{nota1} < 0$ O $\text{nota1} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota1} \geq 0$ Y $\text{nota1} \leq 10$

// Validación Nota 2

Repetir

Escribir "Ingrese la Nota 2 (0 - 10):"

Leer nota2

Si $\text{nota2} < 0$ O $\text{nota2} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota2} \geq 0$ Y $\text{nota2} \leq 10$

// Validación Nota 3

Repetir

Escribir "Ingrese la Nota 3 (0 - 10):"

Leer nota3

Si $\text{nota3} < 0$ O $\text{nota3} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota3} \geq 0$ Y $\text{nota3} \leq 10$



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Calcular promedio
promedio <- (nota1 + nota2 + nota3) / 3

// Determinar estado
Si promedio >= 7 Entonces
    estado <- "Aprobado"
    aprobados <- aprobados + 1
SiNo
    estado <- "Reprobado"
    reprobados <- reprobados + 1
FinSi

// Guardar datos en arreglos
cedulas[i] <- cedula
nombres[i] <- nombre
apellidos[i] <- apellido
notas1[i] <- nota1
notas2[i] <- nota2
notas3[i] <- nota3
promedios[i] <- promedio
estados[i] <- estado

FinPara

// Mostrar resultados
Escribir ""
Escribir "=====
Escribir "      LISTADO DE ESTUDIANTES      "
Escribir "=====

Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer

    Escribir Sin Saltar "Cedula: ", cedulas[i]
        Escribir Sin Saltar " | Estudiante: ", apellidos[i], " ", nombres[i]
        Escribir Sin Saltar " | Notas: [", notas1[i], ", ", notas2[i], ", ", notas3[i], "]"
        Escribir Sin Saltar " | Promedio: ", promedios[i]
```



Escribir Sin Saltar " | Estado: ", estados[i]

FinPara

Escribir ""

Escribir "=====

Escribir "RESUMEN DE RENDIMIENTO"

Escribir "Total Estudiantes Aprobados: ", aprobados

Escribir "Total Estudiantes Reprobados: ", reprobados

Escribir "=====

FinAlgoritmo

2.11.5.6 Diagrama de flujo





2.11.5.7 Codificación C++

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip> // Para dar formato a los decimales (.precision)

using namespace std;

class Estudiante {
private:
    // Atributos privados
    string cedula;
    string nombre;
    string apellido;
    double nota1, nota2, nota3;
    double promedio;
    string estado;

public:
    // Constructor
    Estudiante(string _cedula, string _nombre, string _apellido, double _n1, double _n2, double
_n3) {
        cedula = _cedula;
        nombre = _nombre;
        apellido = _apellido;
        nota1 = _n1;
        nota2 = _n2;
        nota3 = _n3;
        calcularPromedio(); // Se calcula automáticamente al instanciar
        determinarEstado(); // Se determina el estado automáticamente
    }
}
```



```
// Métodos Get y Set

string getCedula() { return cedula; }
void setCedula(string c) { cedula = c; }

string getNombre() { return nombre; }
void setNombre(string n) { nombre = n; }

string getApellido() { return apellido; }
void setApellido(string a) { apellido = a; }

double getNota1() { return nota1; }
void setNota1(double n1) { nota1 = n1; calcularTodo(); }

double getNota2() { return nota2; }
void setNota2(double n2) { nota2 = n2; calcularTodo(); }

double getNota3() { return nota3; }
void setNota3(double n3) { nota3 = n3; calcularTodo(); }

double getPromedio() { return promedio; }
string getEstado() { return estado; }

// Métodos de cálculo internos
void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

void determinarEstado() {
    if (promedio >= 7.0) {
        estado = "Aprobado";
    } else {
        estado = "Reprobado";
    }
}

// Función auxiliar si se modifican notas mediante setters
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
void calcularTodo() {
    calcularPromedio();
    determinarEstado();
}

// Método para mostrar la información del estudiante
void mostrarInformacion() {
    cout << "Cedula: " << cedula
        << " | Estudiante: " << apellido << " " << nombre
        << " | Notas: [" << nota1 << " , " << nota2 << " , " << nota3 << "]"
        << " | Promedio: " << fixed << setprecision(2) << promedio
        << " | Estado: " << estado << endl;
}

};

// Función auxiliar para validar que la nota esté entre 0 y 10
double pedirNotaValidada(string numeroNota) {
    double nota;
    while (true) {
        cout << "Ingrese la " << numeroNota << " (0 - 10): ";
        cin >> nota;
        if (nota >= 0.0 && nota <= 10.0) {
            return nota;
        }
        cout << "¡Nota invalida! Debe estar entre 0 y 10. Intente de nuevo.\n";
    }
}

int main() {
    vector<Estudiante> listaEstudiantes;
    int cantidadEstudiantes = 5; // Requerimiento mínimo

    cout << "=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===" << endl;

    for (int i = 0; i < cantidadEstudiantes; i++) {
        string cedula, nombre, apellido;
        double n1, n2, n3;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "\n--- Datos del Estudiante " << (i + 1) << " ---" << endl;
cout << "Ingrese Cedula: ";
cin >> cedula;
cout << "Ingrese Nombre: ";
cin >> nombre;
cout << "Ingrese Apellido: ";
cin >> apellido;

// Validar cada nota mediante la función auxiliar
n1 = pedirNotaValidada("Nota 1");
n2 = pedirNotaValidada("Nota 2");
n3 = pedirNotaValidada("Nota 3");

// Crear el objeto y guardarlo en el vector
Estudiante nuevoEstudiante(cedula, nombre, apellido, n1, n2, n3);
listaEstudiantes.push_back(nuevoEstudiante);
}

// Reportes finales
cout << "\n===== " << endl;
cout << "      LISTADO DE ESTUDIANTES      " << endl;
cout << "===== " << endl;

int aprobados = 0;
int reprobados = 0;

for (int i = 0; i < listaEstudiantes.size(); i++) {
    listaEstudiantes[i].mostrarInformacion();

    if (listaEstudiantes[i].getEstado() == "Aprobado") {
        aprobados++;
    } else {
        reprobados++;
    }
}
}
```



```
cout << "\n===== " << endl;
cout << "RESUMEN DE RENDIMIENTO" << endl;
cout << "Total Estudiantes Aprobados: " << aprobados << endl;
cout << "Total Estudiantes Reprobados: " << reprobados << endl;
cout << "===== " << endl;

return 0;
}
```

2.11.5.8 Codificación Java

```
import java.util.Scanner;

// _____
// CLASE ESTUDIANTE
// _____

public class SistemasEstudiante {

    // Atributos privados: solo accesibles dentro de la clase
    private String cedula;
    private String nombre;
    private String apellido;
    private double nota1, nota2, nota3;
    private double promedio;
    private String estado;

    // — Constructor: inicializa el objeto con los datos recibidos —
    public SistemasEstudiante(String cedula, String nombre, String apellido,
                               double nota1, double nota2, double nota3) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
        calcularPromedio(); // Se calcula automáticamente al crear el objeto
        determinarEstado();
    }
}
```



// _____



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// MÉTODO MAIN: punto de entrada del programa
// _____

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    final int TOTAL = 5;           // Mínimo 5 estudiantes
    SistemasEstudiante[] lista = new SistemasEstudiante[TOTAL]; // Arreglo de objetos

    System.out.println("=== REGISTRO DE ESTUDIANTES ===\n");

    // — Registro de cada estudiante —
    for (int i = 0; i < TOTAL; i++) {
        System.out.println("--- Estudiante " + (i + 1) + " ---");

        System.out.print(" Cédula  : "); String ced = sc.next();
        System.out.print(" Nombre  : "); String nom = sc.next();
        System.out.print(" Apellido : "); String ape = sc.next();

        double n1 = pedirNota(sc, "Nota 1");
        double n2 = pedirNota(sc, "Nota 2");
        double n3 = pedirNota(sc, "Nota 3");
        System.out.println();

        // Se crea el objeto y se guarda en el arreglo
        lista[i] = new SistemasEstudiante(ced, nom, ape, n1, n2, n3);
    }

    // — Listado completo —
    System.out.println("\n=== LISTADO DE ESTUDIANTES ===");
    int aprobados = 0, reprobados = 0;

    for (SistemasEstudiante e : lista) {
        e.mostrarInfo();
        // Conteo de aprobados y reprobados
        if (e.getEstado().equals("Aprobado")) aprobados++;
        else reprobados++;
    }
}
```



```
// — Resumen final —  
System.out.println("\n=== RESUMEN ===");  
System.out.println(" Aprobados : " + aprobados);  
System.out.println(" Reprobados : " + reprobados);  
  
sc.close();  
}  
  
// —————  
// Pide una nota válida (0–10), repite si es inválida  
// —————  
private static double pedirNota(Scanner sc, String label) {  
    double nota;  
    do {  
        System.out.print(" " + label + " (0-10): ");  
        nota = sc.nextDouble();  
        if (nota < 0 || nota > 10)  
            System.out.println(" ⚠ Nota inválida. Ingrese un valor entre 0 y 10.");  
    } while (nota < 0 || nota > 10); // Repite hasta obtener valor válido  
    return nota;  
}  
}
```

2.11.5.9 Prueba de Escritorio

```
=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===  
  
--- Datos del Estudiante 1 ---  
Ingrese Cedula: 1850342724  
Ingrese Nombre: Diego  
Ingrese Apellido: Balarezo  
Ingrese la Nota 1 (0 - 10): 7  
Ingrese la Nota 2 (0 - 10): 9  
Ingrese la Nota 3 (0 - 10): 9.5  
  
--- Datos del Estudiante 2 ---  
Ingrese Cedula: 1805695826  
Ingrese Nombre: Mateo  
Ingrese Apellido: Vargas  
Ingrese la Nota 1 (0 - 10): 8  
Ingrese la Nota 2 (0 - 10): 7  
Ingrese la Nota 3 (0 - 10): 10  
  
--- Datos del Estudiante 3 ---  
Ingrese Cedula: 1800083416  
Ingrese Nombre: Patricio  
Ingrese Apellido: Ramirez  
Ingrese la Nota 1 (0 - 10): 9  
Ingrese la Nota 2 (0 - 10): 9  
Ingrese la Nota 3 (0 - 10): 6  
  
--- Datos del Estudiante 4 ---  
Ingrese Cedula: 1856478934  
Ingrese Nombre: Ramiro  
Ingrese Apellido: Jacome  
Ingrese la Nota 1 (0 - 10): 7  
Ingrese la Nota 2 (0 - 10): 6  
Ingrese la Nota 3 (0 - 10): 8
```



```
--- Datos del Estudiante 5 ---
Ingrese Cedula: 1857362659
Ingrese Nombre: Sebastian
Ingrese Apellido: Lopez
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9

=====
LISTADO DE ESTUDIANTES
=====
Cedula: 1850342724 | Estudiante: Balarezo Diego | Notas: [7, 9, 9.5] | Promedio: 8.50 | Estado: Aprobado
Cedula: 1805695826 | Estudiante: Vargas Mateo | Notas: [8.00, 7.00, 10.00] | Promedio: 8.33 | Estado: Aprobado
Cedula: 1800083416 | Estudiante: Ramirez Patricio | Notas: [9.00, 9.00, 6.00] | Promedio: 8.00 | Estado: Aprobado
Cedula: 1856478934 | Estudiante: Jacome Ramiro | Notas: [7.00, 6.00, 8.00] | Promedio: 7.00 | Estado: Aprobado
Cedula: 1857362659 | Estudiante: Lopez Sebastian | Notas: [8.00, 9.00, 9.00] | Promedio: 8.67 | Estado: Aprobado

=====
RESUMEN DE RENDIMIENTO
Total Estudiantes Aprobados: 5
Total Estudiantes Reprobados: 0
=====
PS C:\Users\disba\Documents\Diegui\UTA Primer Semestre\VSCodeProjects> |
```

2.11.6 Ejercicio 6

2.11.6.1 Enunciado

Control de Asistencia

Guardar la asistencia de estudiantes.

Requerimientos:

- Registrar nombre, fecha y estado.
- Mostrar historial.
- Contar faltas.

Aplicar:

- Archivos
- Validaciones
- Ciclos



2.11.6.2 Análisis del Problema

El problema consiste en desarrollar un sistema para el control de asistencia estudiantil con persistencia de datos. Para resolverlo, el programa debe capturar de forma secuencial el nombre del estudiante, la fecha del registro y su estado de asistencia (asistió o faltó). Se requiere la aplicación de estructuras de control cíclicas para sostener un menú interactivo y forzar la captura correcta de datos, implementando validaciones estrictas que filtren el formato de la fecha (DD/MM/AAAA) y restrinjan el estado únicamente a caracteres válidos (A o F). Finalmente, los datos deben ser almacenados permanentemente en un archivo de texto plano sin sobrescribir la información previa, permitiendo al sistema leer dicho archivo línea por línea en cualquier momento para reconstruir el historial completo en pantalla y computar de manera matemática el desglose total de las faltas registradas.

2.11.6.3 Declaración de Variables

Variable / Constante	Tipo de Dato	Ámbito/ Ubicación	Descripción Funcional
NOMBRE_ARCHIVO	Cadena (String)	Global (Constante)	Almacena el nombre físico del archivo de texto ("asistencia.txt") en el disco.
opcion	Entero (int)	Menú Principal (main)	Almacena la opción seleccionada por el usuario para controlar el flujo del ciclo do-while.
nombre	Cadena (String)	Subproceso Registro	Guarda el nombre completo del estudiante ingresado por teclado (permite espacios).



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



fecha	Cadena (String)	Subproceso Registro / Validación	Almacena la fecha de la asistencia una vez que ha superado el formato numérico DD/MM/AAAA.
estado	Carácter (char)	Subproceso Registro / Validación	Almacena la condición del alumno en mayúscula ('A' = Asistió, 'F' = Faltó).
linea	Cadena (String)	Subproceso Historial	Almacena de forma temporal el texto completo de una fila leída desde el archivo.
partes	Arreglo de Cadenas	Subproceso Historial	Vector que guarda los datos del alumno tras segmentar la línea usando el delimitador ().
totalRegistros	Entero (int)	Subproceso Historial	Contador acumulativo que registra la cantidad total de asistencias procesadas en el archivo.
totalFaltas	Entero (int)	Subproceso Historial	Contador estadístico que se incrementa únicamente si el estado leído es igual a 'F'.
i	Entero (int)	Local (Validación / Ciclos)	Variable de control usada como índice para recorrer cadenas de texto o estructuras repetitivas.



2.11.6.4 Algoritmo Paso a Paso

Inicio

Ciclo Menú: Mostrar opciones (1. Registrar, 2. Historial, 3. Salir y leer selección.

Si elige 1 (Registrar):

- Solicitar el nombre del estudiante.
- Ciclo Validar Fecha: Pedir la fecha hasta que cumpla el formato DD/MM/AAAA.
- Ciclo Validar Estado: Pedir el estado y forzar a que sea solo A (Asistió) o F (Faltó).
- Abrir el archivo, guardar los datos en una línea separados por | y cerrarlo

Si elige 2 (Historial):

- Poner contadores de registros y faltas en 0. Si el archivo no existe, avisar y volver al menú.
- Ciclo Leer Archivo: Leer línea por línea hasta el final:
- Separar los datos usando el delimitador |.
- Si el estado es F, sumar 1 al contador de faltas. Sumar 1 al total de registros.
- Mostrar la información del estudiante ordenada en pantalla.
- Mostrar el total de registros procesados y las faltas contabilizadas. Cerrar archivo.

Si elige 3 (Salir): Mostrar mensaje de despedida.

Repetir el menú mientras la opción no sea 3.

Fin

2.11.6.5 Pseudocódigo

Algoritmo Control_Asiencia

// 1. DIMENSIÓN DE LOS ARREGLOS

Definir MAX_REGISTROS Como Entero

MAX_REGISTROS <- 100

Definir archivoNombres, archivoFechas, archivoEstados Como Cadena

Dimension archivoNombres[MAX_REGISTROS]

Dimension archivoFechas[MAX_REGISTROS]

Dimension archivoEstados[MAX_REGISTROS]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// 2. VARIABLES DE CONTROL

Definir totalRegistros, opcion, totalFaltas, i Como Entero

Definir nombre, fecha, estado, textoEstado Como Cadena

Definir valida Como Logico

totalRegistros <- 0

// 3. CICLO PRINCIPAL DEL MENÚ (Formato Repetir-Hasta Que)

Repetir

Escribir "=====

Escribir " SISTEMA CONTROL DE ASISTENCIA "

Escribir "=====

Escribir "1. Registrar Asistencia"

Escribir "2. Mostrar Historial y Faltas"

Escribir "3. Salir del programa"

Escribir "-----"

Escribir Sin Saltar "Seleccione una opcion (1-3): "

Leer opcion

Segun opcion Hacer

1:

Si totalRegistros < MAX_REGISTROS Entonces

totalRegistros <- totalRegistros + 1

Escribir ""

Escribir "--- NUEVO REGISTRO DE
ASISTENCIA ---"

Escribir Sin Saltar " Nombre del estudiante: "

Leer nombre

archivoNombres[totalRegistros] <- nombre

// Validación de Fecha

valida <- Falso

Mientras valida = Falso Hacer

Escribir Sin Saltar " Ingrese fecha

(DD/MM/AAAA): "

Leer fecha



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Si Longitud(fecha) = 10 Entonces
    valida <- Verdadero
Sino
    Escribir " [Error] La fecha
debe tener exactamente 10 caracteres."

FinSi
FinMientras
archivoFechas[totalRegistros] <- fecha

// Validación de Estado
valida <- Falso
Mientras valida = Falso Hacer
    Escribir Sin Saltar " Ingrese estado (A
= Asistio, F = Falto): "

    Leer estado
    estado <- Mayusculas(estado)
    Si (estado = "A" O estado = "F") Y
Longitud(estado) = 1 Entonces
        valida <- Verdadero
    Sino
        Escribir " [Error] Estado
invalido. Solo se permite A o F."

    FinSi
FinMientras
archivoEstados[totalRegistros] <- estado

Escribir ""
Escribir ">> Asistencia registrada con exito."
Sino
    Escribir "[Error] Memoria llena."
FinSi

2:
totalFaltas <- 0
Escribir ""
Escribir
```

"=====



Escribir " HISTORIAL DE ASISTENCIA"



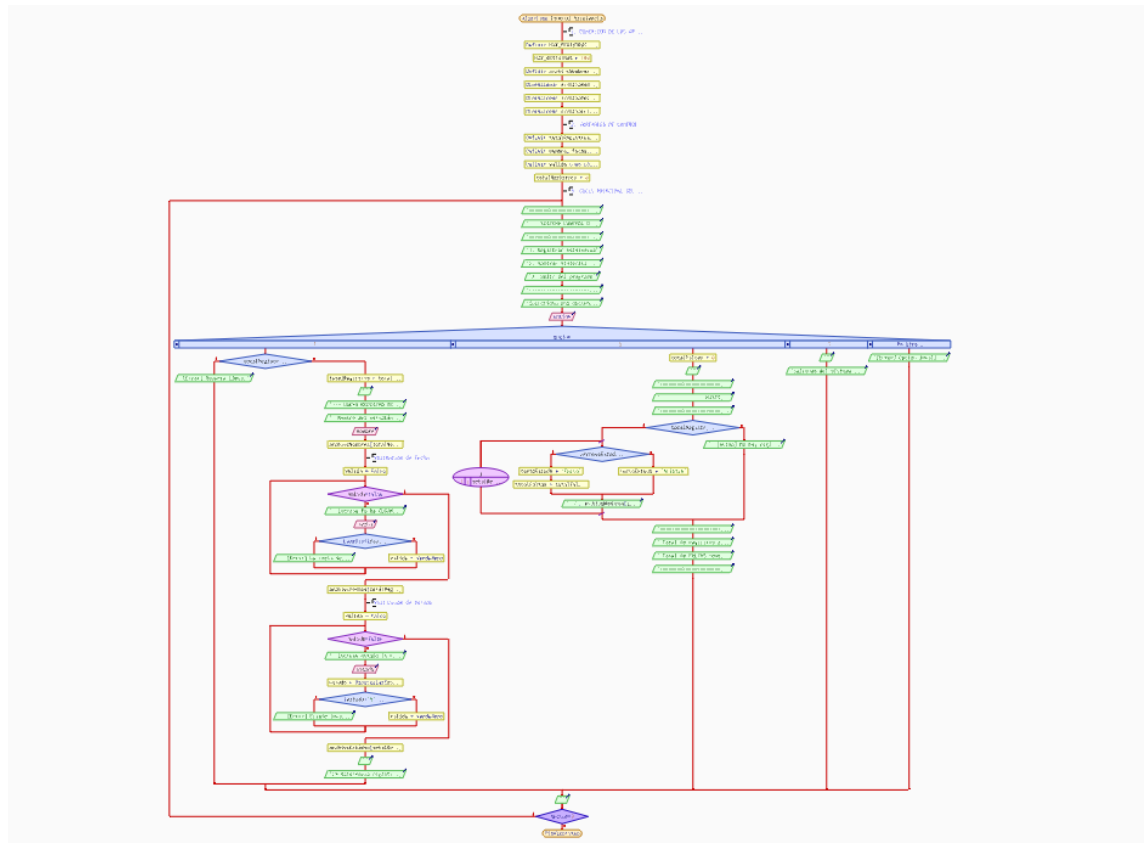
FinSegun

Escribir ""

Hasta Que opcion = 3

FinAlgoritmo

2.11.6.6 Diagrama de flujo



2.11.6.7 Codificación C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
//
```

```
=====
```

```
// FUNCIONES AUXILIARES DE VALIDACIÓN
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



//

=====

=====

// Valida de forma básica que la fecha tenga el formato numérico DD/MM/AAAA

```
bool validarFecha(const string& fecha) {  
    if (fecha.length() != 10) return false;  
    if (fecha[2] != '/' || fecha[5] != '/') return false;  
  
    // Verifica que el resto sean números  
    for (int i = 0; i < 10; i++) {  
        if (i != 2 && i != 5) {  
            if (fecha[i] < '0' || fecha[i] > '9') return false;  
        }  
    }  
    return true;  
}
```

// Solicita la fecha y fuerza al ciclo a repetirse hasta que sea válida

```
string solicitarFecha() {  
    string fecha;  
    while (true) {  
        cout << " Ingrese fecha (DD/MM/AAAA): ";  
        cin >> fecha;  
        if (validarFecha(fecha)) {  
            return fecha;  
        }  
        cout << " [Error] Formato de fecha incorrecto. Use DD/MM/AAAA.\n";  
    }  
}
```

// Fuerza al usuario a ingresar únicamente 'A' o 'F'

```
char solicitarEstado() {  
    char estado;  
    while (true) {  
        cout << " Ingrese estado (A = Asistio, F = Falto): ";  
        cin >> estado;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
estado = toupper(estado); // Lo convierte a mayúscula automáticamente
if (estado == 'A' || estado == 'F') {
    return estado;
}
cout << " [Error] Estado invalido. Solo se permite A o F.\n";
}
}

//
=====
=====

// FUNCIONES DE CONTROL DE ASISTENCIA (MANEJO DE ARCHIVOS)
//
=====
=====

// REQUERIMIENTO 1: Registrar nombre, fecha y estado en el archivo
void registrarAsistencia() {
    // ios::app abre el archivo en modo "append" para añadir texto al final sin borrar lo
    anterior
    ofstream archivoEscritura("asistencia.txt", ios::app);

    if (!archivoEscritura.is_open()) {
        cout << "Error: No se pudo abrir el archivo para guardar datos.\n";
        return;
    }

    string nombre, fecha;
    char estado;

    cout << "\n--- NUEVO REGISTRO DE ASISTENCIA ---\n";
    cout << " Nombre del estudiante: ";
    cin.ignore(); // Limpia el buffer antes de leer una cadena completa con espacios
    getline(cin, nombre);

    fecha = solicitarFecha();
    estado = solicitarEstado();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Guardamos los datos separados por un caracter delimitador (pipe '|') para facilitar la
lectura posterior
```

```
archivoEscritura << nombre << "|" << fecha << "|" << estado << "\n";
archivoEscritura.close();
```

```
cout << "\n>> Asistencia registrada con exito en el sistema.\n";
}
```

```
// REQUERIMIENTO 2 y 3: Mostrar historial completo y contar las faltas totales
```

```
void mostrarHistorialYFaltas() {
```

```
    ifstream archivoLectura("asistencia.txt");
```

```
    if (!archivoLectura.is_open()) {
```

```
        cout << "\n[Aviso] No hay registros previos de asistencia (El archivo aun no
existe).\n";
```

```
        return;
```

```
    }
```

```
    string linea;
```

```
    int totalFaltas = 0;
```

```
    int totalRegistros = 0;
```

```
                                cout <<
```

```
"\n===== \n";
```

```
    cout << "            HISTORIAL DE ASISTENCIA\n";
```

```
                                cout <<
```

```
"===== \n";
```

```
    cout << " %-20s %-15s %-10s\n";
```

```
    cout << " ----- \n";
```

```
// Ciclo para leer el archivo línea por línea hasta el final
```

```
while (getline(archivoLectura, linea)) {
```

```
    if (linea.empty()) continue;
```

```
    stringstream ss(linea);
```

```
    string nombre, fecha, estadoStr;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Descomponemos la línea usando el delimitador '|'
getline(ss, nombre, '|');
getline(ss, fecha, '|');
getline(ss, estadoStr, '|');

char estado = estadoStr[0];

// Traducimos el caracter plano a un texto legible para el usuario
string textoEstado = (estado == 'A') ? "Asistio" : "Falto";

// Si es una falta, incrementamos el contador
if (estado == 'F') {
    totalFaltas++;
}

// Imprimimos la fila formateada de forma ordenada
cout << " " << nombre;
for (size_t i = nombre.length(); i < 22; i++) cout << " ";
cout << fecha << " " << textoEstado << "\n";

totalRegistros++;
}

archivoLectura.close();

cout << "\n";
cout << " Total de registros procesados: " << totalRegistros << "\n";
cout << " Total de FALTAS contabilizadas: " << totalFaltas << "\n";

cout << "\n";
}

//
=====
=====
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// PROGRAMA PRINCIPAL

//

=====

=====

```
int main() {
    int opcion;

    // Ciclo principal del menú de control de asistencia
    do {
        cout << "\n===== \n";
        cout << "  SISTEMA CONTROL DE ASISTENCIA \n";
        cout << "===== \n";
        cout << "1. Registrar Asistencia\n";
        cout << "2. Mostrar Historial y Faltas\n";
        cout << "3. Salir del programa\n";
        cout << "----- \n";
        cout << "Seleccione una opcion (1-3): ";

        // Validación para evitar bucles infinitos si ingresan letras en el menú
        if (!(cin >> opcion)) {
            cout << "Opcion no valida. Intente de nuevo.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (opcion) {
            case 1:
                registrarAsistencia();
                break;
            case 2:
                mostrarHistorialYFaltas();
                break;
            case 3:
                cout << "\nSaliendo del sistema de asistencia... ¡Hasta luego!\n";
                break;
            default:
```



```
        cout << "[Error] La opcion elegida no existe. Intente con 1, 2 o 3.\n";
    }
} while (opcion != 3); // Rompe el ciclo cuando la opción es 3

return 0;
}
```

2.11.6.8 Codificación Java

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

//
=====

// CLASE PRINCIPAL: SistemaAsistencia
//
=====

public class SistemaAsistencia {

    private static final String NOMBRE_ARCHIVO = "asistencia.txt";
    private static final Scanner teclado = new Scanner(System.in);

    // Valida de forma básica que la fecha tenga el formato numérico DD/MM/AAAA
    public static boolean validarFecha(String fecha) {
        if (fecha.length() != 10) return false;
        if (fecha.charAt(2) != '/' || fecha.charAt(5) != '/') return false;

        // Verifica que los demás caracteres sean dígitos
        for (int i = 0; i < 10; i++) {
            if (i != 2 && i != 5) {
                if (fecha.charAt(i) < '0' || fecha.charAt(i) > '9') return false;
            }
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
}  
}  
return true;  
}  
  
// Solicita la fecha y fuerza al ciclo a repetirse hasta que sea válida  
public static String solicitarFecha() {  
    String fecha;  
    while (true) {  
        System.out.print(" Ingrese fecha (DD/MM/AAAA): ");  
        fecha = teclado.next();  
        if (validarFecha(fecha)) {  
            return fecha;  
        }  
        System.out.println(" [Error] Formato de fecha incorrecto. Use  
DD/MM/AAAA.");  
    }  
}  
  
// Fuerza al usuario a ingresar únicamente 'A' o 'F'  
public static char solicitarEstado() {  
    String entrada;  
    while (true) {  
        System.out.print(" Ingrese estado (A = Asistio, F = Falto): ");  
        entrada = teclado.next().toUpperCase(); // Convierte a mayúscula  
        automáticamente  
        if (entrada.length() == 1 && (entrada.charAt(0) == 'A' || entrada.charAt(0) ==  
'F')) {  
            return entrada.charAt(0);  
        }  
        System.out.println(" [Error] Estado invalido. Solo se permite A o F.");  
    }  
}  
  
// REQUERIMIENTO 1: Registrar nombre, fecha y estado en el archivo  
public static void registrarAsistencia() {  
    // El parámetro true en FileWriter habilita el modo "append" (añadir al final)
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
try (FileWriter fw = new FileWriter(NOMBRE_ARCHIVO, true);
    PrintWriter archivoEscritura = new PrintWriter(fw)) {

    System.out.println("\n--- NUEVO REGISTRO DE ASISTENCIA ---");
    System.out.print(" Nombre del estudiante: ");
    teclado.nextLine(); // Limpia el buffer del Scanner
    String nombre = teclado.nextLine();

    String fecha = solicitarFecha();
    char estado = solicitarEstado();

    // Guardamos los datos separados por la barra vertical '|'
    archivoEscritura.println(nombre + "|" + fecha + "|" + estado);
    System.out.println("\n>> Asistencia registrada con éxito en el archivo.");

} catch (IOException e) {
    System.out.println("Error al intentar escribir en el archivo: " + e.getMessage());
}

// REQUERIMIENTO 2 y 3: Mostrar historial completo y contar las faltas totales
public static void mostrarHistorialYFaltas() {
    File archivo = new File(NOMBRE_ARCHIVO);

    // Comprobamos de antemano si el archivo físico existe en el disco
    if (!archivo.exists()) {
        System.out.println("\n[Aviso] No hay registros previos de asistencia (El archivo
aun no existe).");
        return;
    }

    int totalFaltas = 0;
    int totalRegistros = 0;

    System.out.println("\n=====
=====");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("        HISTORIAL DE ASISTENCIA");
```

```
System.out.println("=====");  
=====);
```

```
System.out.printf(" %-22s %-15s %-10s\n", "Estudiante", "Fecha", "Estado");
```

```
System.out.println(" -----");
```

```
try (FileReader fr = new FileReader(archivo);
```

```
    BufferedReader archivoLectura = new BufferedReader(fr)) {
```

```
    String linea;
```

```
    // Ciclo para leer el archivo línea por línea hasta llegar al final (null)
```

```
    while ((linea = archivoLectura.readLine()) != null) {
```

```
        if (linea.trim().isEmpty()) continue;
```

```
        // Dividimos la cadena usando la barra vertical como separador
```

```
        // Usamos \\ porque en Java split requiere una expresión regular y el | es un  
caracter reservado
```

```
        String[] partes = linea.split("\\|");
```

```
        if (partes.length == 3) {
```

```
            String nombre = partes[0];
```

```
            String fecha = partes[1];
```

```
            char estado = partes[2].charAt(0);
```

```
            String textoEstado = (estado == 'A') ? "Asistio" : "Falto";
```

```
            if (estado == 'F') {
```

```
                totalFaltas++;
```

```
            }
```

```
            System.out.printf(" %-22s %-15s %-10s\n", nombre, fecha, textoEstado);
```

```
            totalRegistros++;
```

```
        }
```

```
    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("=====
=====");
    System.out.println(" Total de registros procesados: " + totalRegistros);
    System.out.println(" Total de FALTAS contabilizadas: " + totalFaltas);

System.out.println("=====
=====");

    } catch (IOException e) {
        System.out.println("Error al intentar leer el archivo: " + e.getMessage());
    }
}

//

=====
=====

// PROGRAMA PRINCIPAL
//

=====
=====

public static void main(String[] args) {
    int opcion = 0;

    // Ciclo principal del menú de control de asistencia
    do {
        System.out.println("\n=====");
        System.out.println(" SISTEMA CONTROL DE ASISTENCIA ");
        System.out.println("=====");
        System.out.println("1. Registrar Asistencia");
        System.out.println("2. Mostrar Historial y Faltas");
        System.out.println("3. Salir del programa");
        System.out.println("-----");
        System.out.print("Seleccione una opcion (1-3): ");

        // Validación para evitar bucles infinitos por ingreso de letras
        if (teclado.hasNextInt()) {
```



```
opcion = teclado.nextInt();

switch (opcion) {
    case 1:
        registrarAsistencia();
        break;
    case 2:
        mostrarHistorialYFaltas();
        break;
    case 3:
        System.out.println("\nSaliendo del sistema de asistencia... ¡Hasta
luego!");
        break;
    default:
        System.out.println("[Error] La opcion elegida no existe. Intente con 1, 2
o 3.");
}
} else {
    System.out.println("[Error] Entrada no valida. Debe ingresar un numero
entero.");
    teclado.next(); // Descarta la entrada incorrecta del buffer
    opcion = 0; // Reinicia la opción para que continúe el ciclo
}

} while (opcion != 3);

teclado.close(); // Cerramos el recurso del teclado
}
}
```

2.11.6.9 Prueba de Escritorio

opcion	totalRegistros	nombre	fecha	estado	totalFaltas	Salida en Pantalla



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



1	1	"Paulo"	"22/05/2026"	"A"	0	>> Asistencia registrada con éxito.
1	2	"Carolina"	"23/05/2026"	"F"	0	>> Asistencia registrada con éxito.
1	3	"Jhon"	"24/05/2026"	"F"	0	>> Asistencia registrada con éxito.
2	3	—	—	—	2	Total registros: 3 / Total faltas: 2
3	3	—	—	—	2	Saliendo del sistema...

2.11.7 Ejercicio 7

2.11.7.1 Enunciado

Cajero Automático Básico

Simular un cajero automático utilizando archivos.

Requerimientos:

- Guardar usuario y saldo.
- Consultar saldo.
- Depositar y retirar dinero.

Aplicar:

- POO
- Archivos
- Métodos

2.11.7.2 Análisis del Problema



El problema consiste en desarrollar un sistema básico de cajero automático utilizando programación orientada a objetos en C++ y Java. El programa debe permitir registrar un usuario junto con su saldo inicial y ofrecer funciones básicas como consultar saldo, depositar dinero y retirar dinero.

Además, la información del usuario y el saldo final deben guardarse en un archivo de texto para mantener un registro de las operaciones realizadas. Para resolver este ejercicio se aplican clases, objetos, métodos y manejo de archivos, permitiendo organizar mejor el código y facilitar el control de las operaciones del cajero automático.

2.11.7.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
usuario	string	String	Almacena el nombre del usuario del cajero.
saldo	double	double	Guarda el saldo disponible en la cuenta.
saldoInicial	double	double	Almacena el saldo inicial ingresado por el usuario.
monto	double	double	Guarda el monto que se desea depositar o retirar.
opcion	int	int	Permite seleccionar las opciones del menú del cajero automático.
cuenta	Cuenta	Cuenta	Objeto que almacena la información y operaciones de la cuenta.
archivo	ofstream	FileWriter	Permite guardar la información en el archivo cajero.txt.
entrada	cin	Scanner	Permite ingresar datos desde el teclado.

2.11.7.4 Algoritmo Paso a Paso

Inicio



Definir una clase llamada Cuenta con atributos para usuario y saldo

Definir variables para el saldo inicial, monto y opción del menú

Solicitar al usuario el nombre de la cuenta y el saldo inicial

Guardar la información ingresada en el objeto cuenta

Crear y abrir el archivo cajero.txt para guardar la información

Mostrar un menú con las opciones consultar saldo, depositar dinero, retirar dinero y salir

Solicitar al usuario una opción del menú

Si la opción es consultar saldo mostrar el saldo disponible en la cuenta

Si la opción es depositar dinero solicitar el monto a depositar y sumar el valor al saldo actual

Si la opción es retirar dinero solicitar el monto a retirar y verificar si existe saldo suficiente antes de descontarlo

Repetir el menú hasta que el usuario seleccione la opción salir

Guardar el nombre del usuario y el saldo final en el archivo cajero.txt

Cerrar el archivo para guardar correctamente la información

Fin

2.11.7.5 Pseudocódigo

Algoritmo CajeroAutomatico

```
// Declaración de variables
Definir usuario Como Cadena
Definir saldo, saldoInicial, monto Como Real
Definir opcion Como Entero

// Solicitar datos iniciales
Escribir "Ingrese el nombre del usuario: "
Leer usuario

Escribir "Ingrese el saldo inicial: "
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Leer saldoInicial

// Guardar saldo inicial

saldo <- saldoInicial

// Menú principal

Repetir

// Mostrar opciones del cajero

Escribir "===== CAJERO AUTOMATICO ====="

Escribir "1. Consultar saldo"

Escribir "2. Depositar dinero"

Escribir "3. Retirar dinero"

Escribir "4. Salir"

Escribir "Seleccione una opcion: "

Leer opcion

Segun opcion Hacer

1:

// Mostrar saldo disponible

Escribir "Saldo actual: \$", saldo

2:

// Depositar dinero

Escribir "Ingrese el monto a depositar: "

Leer monto

saldo <- saldo + monto

Escribir "Deposito realizado correctamente"

3:

// Retirar dinero



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Ingrese el monto a retirar: "

Leer monto

// Verificar si existe saldo suficiente

Si monto $\leq$ saldo Entonces

saldo $\leftarrow$ saldo - monto

Escribir "Retiro realizado correctamente"

SiNo

Escribir "Saldo insuficiente"

FinSi

4:

// Salir del sistema

Escribir "Saliendo del sistema..."

De Otro Modo:

// Opción inválida

Escribir "Opcion incorrecta"

FinSegun

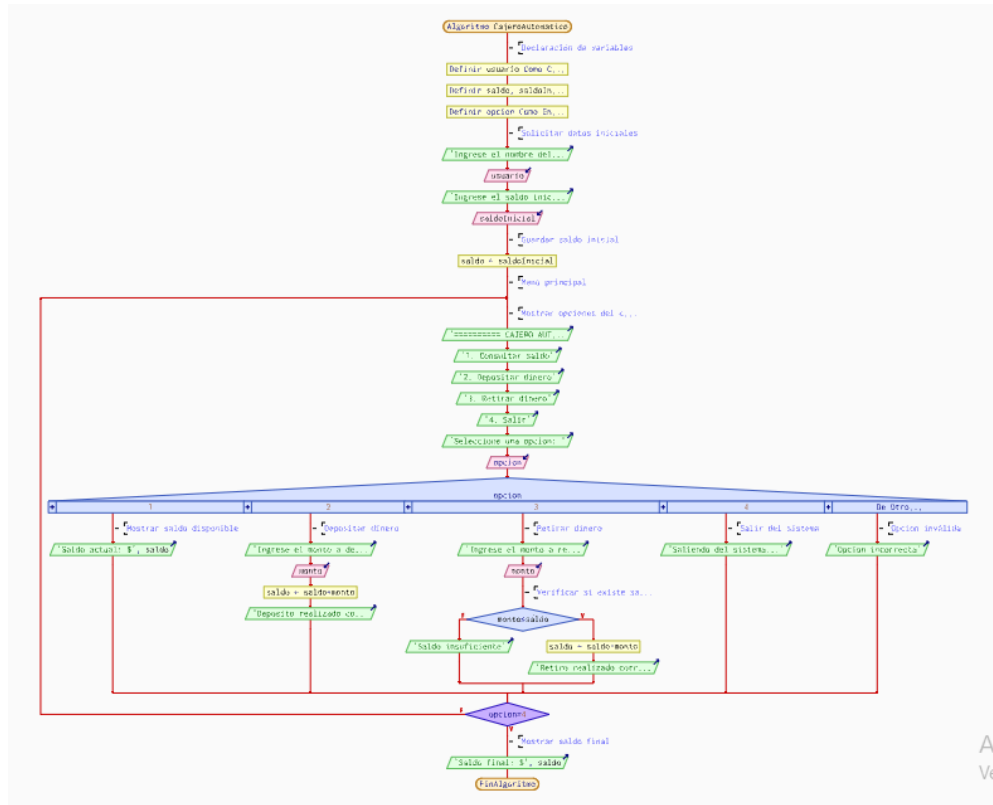
Hasta Que opcion = 4

// Mostrar saldo final

Escribir "Saldo final: \$", saldo

FinAlgoritmo

2.11.7.6 Diagrama de flujo



2.11.7.7 Codificación C++

2.11.7.7.1 Clase Cuenta.h

// Librerías necesarias

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

// Clase Cuenta

```
class Cuenta {
```

// Atributos privados

private:

string usuario;

double saldo;

public:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Constructor vacío
Cuenta() {

    usuario = "";
    saldo = 0;
}

// Métodos SET
void setUsuario(string u) {
    usuario = u;
}

void setSaldo(double s) {
    saldo = s;
}

// Métodos GET
string getUsuario() {
    return usuario;
}

double getSaldo() {
    return saldo;
}

// Método para depositar dinero
void depositar(double monto) {

    saldo += monto;
}

// Método para retirar dinero
void retirar(double monto) {

    // Verificar si tiene suficiente saldo
    if(monto <= saldo) {
```



```
        saldo -= monto;
        cout << "Retiro realizado correctamente" << endl;

    } else {

        cout << "Saldo insuficiente" << endl;
    }
}

// Método para mostrar saldo
void consultarSaldo() {

    cout << "Saldo actual: $" << saldo << endl;
}

};
```

2.11.7.7.2 Clase Main.cpp

```
// Librerías necesarias
#include <iostream>
#include <fstream>
#include "Cuenta.h"

using namespace std;

int main() {

    // Crear objeto cuenta
    Cuenta cuenta;

    // Variables
    string usuario;
    double saldoInicial;
    double monto;
    int opcion;

    // Archivo para guardar datos
    ofstream archivo("cajero.txt");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Ingreso de datos
cout << "Ingrese el nombre del usuario: ";
getline(cin, usuario);

cout << "Ingrese el saldo inicial: ";
cin >> saldoInicial;

// Guardar datos en el objeto
cuenta.setUsuario(usuario);
cuenta.setSaldo(saldoInicial);

// Menú principal
do {

    cout << "\n===== CAJERO AUTOMATICO =====" << endl;
    cout << "1. Consultar saldo" << endl;
    cout << "2. Depositar dinero" << endl;
    cout << "3. Retirar dinero" << endl;
    cout << "4. Salir" << endl;
    cout << "Seleccione una opcion: ";
    cin >> opcion;

    switch(opcion) {

        case 1:

            // Mostrar saldo
            cuenta.consultarSaldo();
            break;

        case 2:

            // Depositar dinero
            cout << "Ingrese el monto a depositar: ";
            cin >> monto;

            cuenta.depositar(monto);
```



```
cout << "Deposito realizado correctamente" << endl;
break;
```

```
case 3:
```

```
// Retirar dinero
cout << "Ingrese el monto a retirar: ";
cin >> monto;
```

```
cuenta.retirar(monto);
break;
```

```
case 4:
```

```
cout << "Saliendo del sistema..." << endl;
break;
```

```
default:
```

```
    cout << "Opcion invalida" << endl;
}
```

```
} while(opcion != 4);
```

```
// Guardar datos en archivo
archivo << "Usuario: " << cuenta.getUsuario() << endl;
archivo << "Saldo final: $" << cuenta.getSaldo() << endl;
```

```
// Cerrar archivo
archivo.close();
```

```
return 0;
```

```
}
```

2.11.7.8 Codificación Java

2.11.7.8.1 Clase Cuenta.java

```
// Clase Cuenta
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public class Cuenta {

    // Atributos privados
    private String usuario;
    private double saldo;

    // Constructor vacío
    public Cuenta() {

        usuario = "";
        saldo = 0;
    }

    // Métodos SET
    public void setUsuario(String usuario) {
        this.usuario = usuario;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    // Métodos GET
    public String getUsuario() {
        return usuario;
    }

    public double getSaldo() {
        return saldo;
    }

    // Método para depositar dinero
    public void depositar(double monto) {

        saldo += monto;
    }
}
```



```
// Método para retirar dinero
public void retirar(double monto) {

    // Verificar saldo disponible
    if(monto <= saldo) {

        saldo -= monto;
        System.out.println("Retiro realizado correctamente");

    } else {

        System.out.println("Saldo insuficiente");
    }
}

// Método para consultar saldo
public void consultarSaldo() {

    System.out.println("Saldo actual: $" + saldo);
}
}
```

2.11.7.8.2 Clase Main.java

```
// Librerías necesarias
import java.util.Scanner;
import java.io.FileWriter;
import java.io.IOException;

public class Main {

    public static void main(String[] args) {

        // Objeto Scanner
        Scanner entrada = new Scanner(System.in);

        // Crear objeto cuenta
        Cuenta cuenta = new Cuenta();
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Variables
String usuario;
double saldoInicial;
double monto;
int opcion;

try {

    // Archivo para guardar datos
    FileWriter archivo = new FileWriter("cajerojava.txt");

    // Ingreso de datos
    System.out.print("Ingrese el nombre del usuario: ");
    usuario = entrada.nextLine();

    System.out.print("Ingrese el saldo inicial: ");
    saldoInicial = entrada.nextDouble();

    // Guardar datos en el objeto
    cuenta.setUsuario(usuario);
    cuenta.setSaldo(saldoInicial);

    // Menú principal
    do {

        System.out.println("\n===== CAJERO AUTOMATICO =====");
        System.out.println("1. Consultar saldo");
        System.out.println("2. Depositar dinero");
        System.out.println("3. Retirar dinero");
        System.out.println("4. Salir");
        System.out.print("Seleccione una opcion: ");
        opcion = entrada.nextInt();

        switch(opcion) {

            case 1:
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Consultar saldo
```

```
cuenta.consultarSaldo();
```

```
break;
```

```
case 2:
```

```
// Depositar dinero
```

```
System.out.print("Ingrese el monto a depositar: ");
```

```
monto = entrada.nextDouble();
```

```
cuenta.depositar(monto);
```

```
System.out.println("Deposito realizado correctamente");
```

```
break;
```

```
case 3:
```

```
// Retirar dinero
```

```
System.out.print("Ingrese el monto a retirar: ");
```

```
monto = entrada.nextDouble();
```

```
cuenta.retirar(monto);
```

```
break;
```

```
case 4:
```

```
System.out.println("Saliendo del sistema...");
```

```
break;
```

```
default:
```

```
System.out.println("Opcion invalida");
```

```
}
```

```
} while(opcion != 4);
```



```
// Guardar datos en archivo
archivo.write("Usuario: " + cuenta.getUsuario() + "\n");
archivo.write("Saldo final: $" + cuenta.getSaldo() + "\n");

// Cerrar archivo
archivo.close();

} catch(IOException e) {

    System.out.println("Error al crear el archivo");
}

// Cerrar Scanner
entrada.close();
}
}
```

2.11.7.9 Prueba de Escritorio

2.11.7.9.1 En C++

```
C Cuenta.h BancoMain.cpp x cajero.txt
BancoMain.cpp > main()
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\Usuario\OneDrive\Ape05> cd 'c:\Users\Usuario\OneDrive\Ape05\output'
PS C:\Users\Usuario\OneDrive\Ape05\output> & .\BancoMain.exe
Ingrese el nombre del usuario: Karla
Ingrese el saldo inicial: 100

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 2
Ingrese el monto a depositar: 60
Deposito realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $160

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 3
Ingrese el monto a retirar: 40
Retiro realizado correctamente
```



```
===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $120

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 4
Saliendo del sistema...
PS C:\Users\Usuario\OneDrive\Ape05\output>
```

2.11.7.9.2 En Java

```
Cuenta.java X BancoMain.java cajerojava.txt
Cuenta.java > Cuenta > usuario
2 public class Cuenta {
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\Usuario\OneDrive\Ape05> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDe
ExceptionMessages' '-cp' 'C:\Users\Usuario\AppData\Roaming\Code\User\workspaceStorage\31acb19243734
52fc7c06bf91\redhat.java\jdt_ws\Ape05_d35a3370\bin' 'BancoMain'
Ingrese el nombre del usuario: Juan
Ingrese el saldo inicial: 30

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $30.0

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 2
Ingrese el monto a depositar: 30
Deposito realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $60.0
```



```
===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 3
Ingrese el monto a retirar: 50
Retiro realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $10.0

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 4
Saliendo del sistema...
PS C:\Users\Usuario\OneDrive\Ape05> █
```

2.11.8 Ejercicio 8

2.11.8.1 Enunciado

Biblioteca Virtual

Registrar libros en un archivo.

Requerimientos:

- Guardar código, título, autor y estado.
- Buscar libros.

Mostrar disponibles y prestados.

Aplicar:

- POO
- Archivos
- Métodos

2.11.8.2 Análisis del Problema

El ejercicio nos pide crear un programa el cuál, mediante un menú interactivo permita al usuario registrar libros, buscar un libro que se encuentre registrado, verificar los libros guardados si están en proceso o finalizados, y por último cerrar el programa.

Para ello se debe utilizar clases, archivos, métodos y POO enseñados en clase para realizar el código de manera eficiente.



2.11.8.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
codigo	string	String	Variable para poder guardar los datos de los códigos de los libros
titulo	string	String	Variable para poder guardar los títulos de los libros
autor	string	String	Variable para poder guardar los nombres de los autores de los libros
estado	string	String	Variable para poder guardar el estado ya sea terminado o en proceso de los libros
nombreArchivo	string	String	Variable para guardar el nombre del archivo que se generará con los libros registrados.

2.11.8.4 Algoritmo Paso a Paso

Crear la clase llamada Libro.

Definir los atributos privados codigo, titulo, autor, estado como String.

Inicializar el constructor de la clase Libro.

Definir getters y setters para obtener y guardar la información en las variables correspondientes.

Crear la clase Biblioteca.

Definir el atributo privado nombreArchivo como String.

Inicializar el método registrarLibro para registrar nuevos libros.

Inicializar el método buscarLibro para buscar los libros registrados por su número de código.

Inicializar el método mostrarPorEstado para poder mostrar los libros registrados según su estado ya sea terminado o en proceso.

Inicializar método para imprimir los resultados en consola.

Inicializar el método principal.

Crear objeto llamado miBiblioteca a partir de la clase Biblioteca.



Inicializar un ciclo do-while para que el código se ejecute sin problemas hasta que la opción ingresada sea 4.

Imprimir el menú.

Solicitar la opción del menú.

Registrar libro si la opción 1 es seleccionada.

Buscar libro si la opción 2 es seleccionada.

Mostrar libro por clasificación si la opción 3 es seleccionada.

Terminar el código y cerrar el programa si la opción 4 es seleccionada.

2.11.8.5 Pseudocódigo

Algoritmo BibliotecaVirtual

// Declaración de variables

Definir opcion, cantidadLibros, i, estOpcion Como Entero

Definir criterio Como Cadena

Definir encontrado Como Logico

// Arreglos para almacenar los datos de los libros

Definir codigo, titulo, autor, estado como cadena

Dimension codigo[100]

Dimension titulo[100]

Dimension autor[100]

Dimension estado[100]

cantidadLibros <- 0

Repetir

 Escribir "=====

 Escribir " MENU BIBLIOTECA VIRTUAL"

 Escribir "=====

 Escribir "1. Registrar Libro"

 Escribir "2. Buscar Libro"

 Escribir "3. Mostrar Disponibles y En Proceso"

 Escribir "4. Salir"

 Escribir "Seleccione una opcion:"



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Leer opcion

Segun opcion Hacer

// ===== REGISTRAR LIBRO =====

1:

Escribir "Ingrese el codigo del libro:"

Leer codigo[cantidadLibros]

Escribir "Ingrese el titulo del libro:"

Leer titulo[cantidadLibros]

Escribir "Ingrese el autor del libro:"

Leer autor[cantidadLibros]

Escribir "Estado del libro:"

Escribir "1. Terminado"

Escribir "2. En proceso"

Leer estOpcion

Si estOpcion = 1 Entonces

estado[cantidadLibros] <- "Terminado"

SiNo

estado[cantidadLibros] <- "En proceso"

FinSi

cantidadLibros <- cantidadLibros + 1

Escribir "Libro registrado correctamente."

// ===== BUSCAR LIBRO =====

2:

encontrado <- Falso

Escribir "Ingrese codigo o titulo a buscar:"

Leer criterio



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Para i <- 0 Hasta cantidadLibros - 1 Hacer

Si codigo[i] = criterio O titulo[i] = criterio Entonces

Escribir "=====

Escribir " LIBRO ENCONTRADO"

Escribir "=====

Escribir "Codigo: ", codigo[i]

Escribir "Titulo: ", titulo[i]

Escribir "Autor: ", autor[i]

Escribir "Estado: ", estado[i]

encontrado <- Verdadero

FinSi

FinPara

Si encontrado = Falso Entonces

Escribir "No se encontro ningun libro."

FinSi

// ===== MOSTRAR LIBROS =====

3:

Escribir "=====

Escribir " LIBROS TERMINADOS"

Escribir "=====

Para i <- 0 Hasta cantidadLibros - 1 Hacer

Si estado[i] = "Terminado" Entonces

Escribir Sin Saltar "Codigo: ", codigo[i]

Escribir Sin Saltar " | Titulo: ", titulo[i]

Escribir Sin Saltar " | Autor: ", autor[i]

FinSi

FinPara



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "=====

Escribir " LIBROS EN PROCESO"

Escribir "=====

Para i <- 0 Hasta cantidadLibros - 1 Hacer

Si estado[i] = "En proceso" Entonces

Escribir Sin Saltar "Codigo: ", codigo[i]

Escribir Sin Saltar " | Titulo: ", titulo[i]

Escribir Sin Saltar " | Autor: ", autor[i]

FinSi

FinPara

// ===== SALIR =====

4:

Escribir "Saliendo del sistema..."

De Otro Modo:

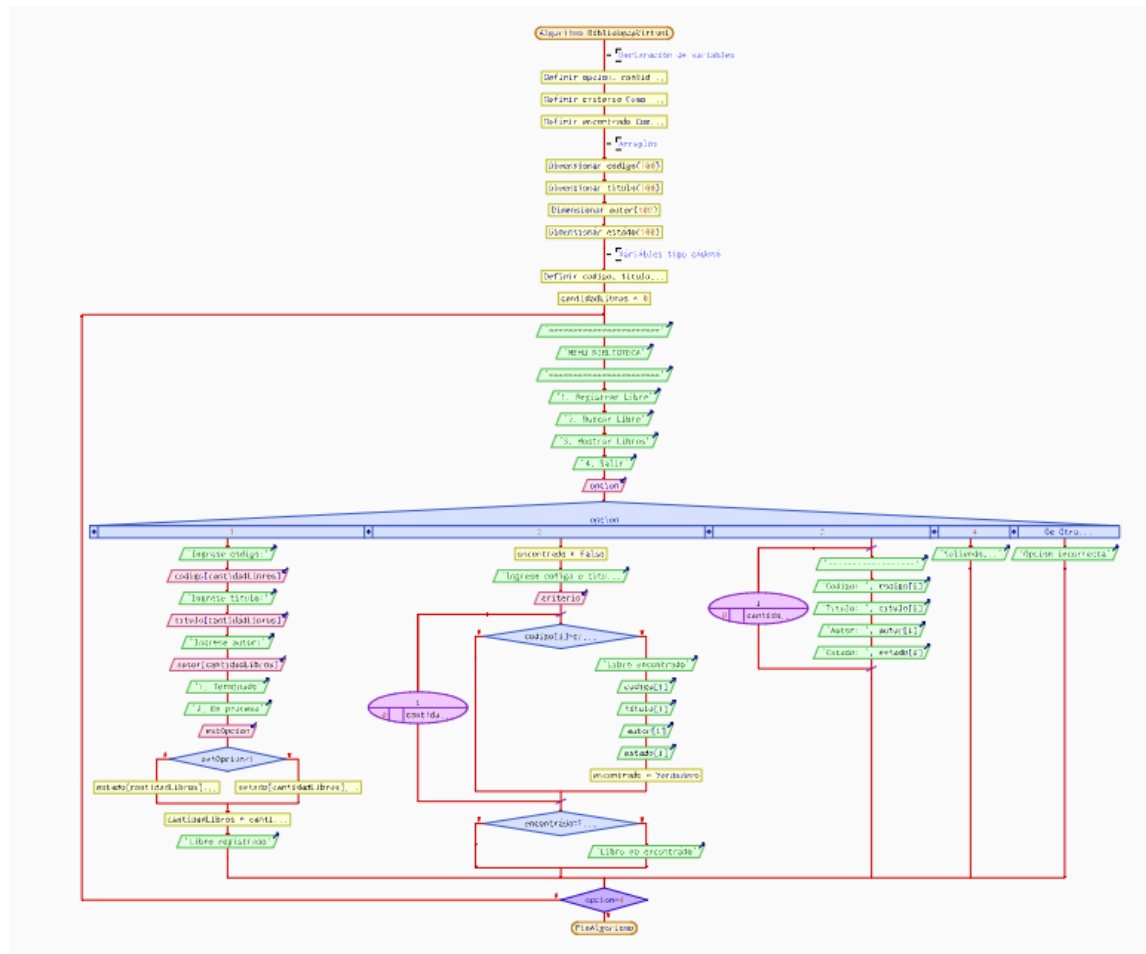
Escribir "Opcion invalida."

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

2.11.8.6 Diagrama de flujo



2.11.8.7 Codificación C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
// === CLASE ENTIDAD ===
```

```
class Libro {
```

```
private:
```

```
    string codigo;
```

```
    string titulo;
```

```
    string autor;
```

```
    string estado; // "Terminado" o "En proceso"
```



public:

```
Libro(string c, string t, string a, string e) {  
    codigo = c;  
    titulo = t;  
    autor = a;  
    estado = e;  
}
```

```
string getCodigo() { return codigo; }  
string getTitulo() { return titulo; }  
string getAutor() { return autor; }  
string getEstado() { return estado; }
```

// Convierte el objeto a una línea de texto para el archivo

```
string toString() {  
    return codigo + "|" + titulo + "|" + autor + "|" + estado;  
}  
};
```

// == CLASE DE LÓGICA / CONTROL ==

```
class Biblioteca {
```

```
private:
```

```
    string nombreArchivo = "biblioteca.txt";
```

public:

// Método para registrar un libro

```
void registrarLibro(Libro libro) {  
    ofstream archivo(nombreArchivo, ios::app); // ios::app para agregar al final  
    if (archivo.is_open()) {  
        archivo << libro.toString() << "\n";  
        archivo.close();  
        cout << "¡Libro registrado con éxito en el archivo!\n";  
    } else {  
        cout << "Error al abrir el archivo para guardar.\n";  
    }  
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Método para buscar un libro por código o título
void buscarLibro(string criterio) {
    ifstream archivo(nombreArchivo);
    string linea;
    bool encontrado = false;

    if (archivo.is_open()) {
        while (getline(archivo, linea)) {
            stringstream ss(linea);
            string cod, tit, aut, est;

            getline(ss, cod, '|');
            getline(ss, tit, '|');
            getline(ss, aut, '|');
            getline(ss, est, '|');

            if (cod == criterio || tit == criterio) {
                cout << "\n--- Libro Encontrado ---" << endl;
                cout << "Código: " << cod << " | Título: " << tit << " | Autor: " << aut << "
| Estado: " << est << endl;
                encontrado = true;
                break;
            }
        }
        archivo.close();
        if (!encontrado) cout << "No se encontró ningún libro que coincida con: " <<
criterio << endl;
    }
}

// Método para mostrar clasificados por estado
void mostrarPorEstado() {
    ifstream archivo(nombreArchivo);
    string linea;

    if (!archivo.is_open()) {
        cout << "Aún no hay libros registrados.\n";
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        return;
    }

    vector<string> terminados;
    vector<string> enProceso;

    while (getline(archivo, linea)) {
        stringstream ss(linea);
        string cod, tit, aut, est;

        getline(ss, cod, '|');
        getline(ss, tit, '|');
        getline(ss, aut, '|');
        getline(ss, est, '|');

        string info = "Cod: " + cod + " - " + tit + " (" + aut + ")";
        if (est == "Terminado") {
            terminados.push_back(info);
        } else {
            enProceso.push_back(info);
        }
    }
    archivo.close();

    cout << "\n=== LIBROS TERMINADOS (Disponibles para lectura completa)
===> << endl;
    for (const string& l : terminados) cout << " • " << l << endl;
    if (terminados.empty()) cout << " (Ninguno)\n";

    cout << "\n=== LIBROS EN PROCESO (Lectura en curso/Prestados) ===> <<
endl;
    for (const string& l : enProceso) cout << " • " << l << endl;
    if (enProceso.empty()) cout << " (Ninguno)\n";
}
};

// === INTERFAZ / MAIN ===
```



```
int main() {
    Biblioteca miBiblioteca;
    int opcion;

    do {
        cout << "\n--- MENU BIBLIOTECA VIRTUAL ---\n";
        cout << "1. Registrar Libro\n";
        cout << "2. Buscar Libro\n";
        cout << "3. Mostrar Disponibles y En Proceso\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;
        cin.ignore(); // Limpiar el buffer del salto de línea

        if (opcion == 1) {
            string cod, tit, aut, est;
            int estOpcion;
            cout << "Código: "; getline(cin, cod);
            cout << "Título: "; getline(cin, tit);
            cout << "Autor: "; getline(cin, aut);
            cout << "Estado (1. Terminado / 2. En proceso): "; cin >> estOpcion;
            est = (estOpcion == 1) ? "Terminado" : "En proceso";

            Libro nuevoLibro(cod, tit, aut, est);
            miBiblioteca.registrarLibro(nuevoLibro);

        } else if (opcion == 2) {
            string criterio;
            cout << "Ingrese código o título a buscar: ";
            getline(cin, criterio);
            miBiblioteca.buscarLibro(criterio);

        } else if (opcion == 3) {
            miBiblioteca.mostrarPorEstado();
        }

    } while (opcion != 4);
}
```



```
return 0;
```

```
}
```

2.11.8.8 Codificación Java

```
import java.io.*;
```

```
import java.util.*;
```

```
// =====
```

```
// CLASE Libro - Representa un libro en la biblioteca
```

```
// =====
```

```
class Libro {
```

```
    private int    codigo;
```

```
    private String titulo;
```

```
    private String autor;
```

```
    private String estado; // "disponible" o "prestado"
```

```
// Constructor
```

```
public Libro(int codigo, String titulo, String autor, String estado) {
```

```
    this.codigo = codigo;
```

```
    this.titulo = titulo;
```

```
    this.autor  = autor;
```

```
    this.estado = estado;
```

```
}
```

```
// Getters
```

```
public int    getCodigo() { return codigo; }
```

```
public String getTitulo() { return titulo; }
```

```
public String getAutor()  { return autor;  }
```

```
public String getEstado() { return estado; }
```

```
// Mostrar info del libro en consola
```

```
public void mostrar() {
```

```
    System.out.printf("%-6d %-30s %-22s %s%n",
```

```
        codigo, titulo, autor, estado);
```

```
}
```

```
// Convertir a línea CSV para guardar en archivo
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public String aTexto() {
    return codigo + "," + titulo + "," + autor + "," + estado;
}

// Crear un Libro desde una línea CSV
public static Libro desdeTexto(String linea) {
    String[] p = linea.split(",", 4);
    return new Libro(Integer.parseInt(p[0]), p[1], p[2], p[3]);
}

// =====
// CLASE Biblioteca - Gestiona la colección de libros
// =====

class Biblioteca {
    private List<Libro> libros = new ArrayList<>();
    private final String ARCHIVO = "libros.txt";

    // Constructor: carga libros existentes
    public Biblioteca() { cargarDesdeArchivo(); }

    // — Cargar todos los libros desde el archivo —————
    private void cargarDesdeArchivo() {
        libros.clear();
        File f = new File(ARCHIVO);
        if (!f.exists()) return;

        try (BufferedReader br = new BufferedReader(new FileReader(f))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                if (!linea.isBlank())
                    libros.add(Libro.desdeTexto(linea));
            }
        } catch (IOException e) {
            System.out.println("[!] Error al leer archivo: " + e.getMessage());
        }
    }
}
```



```
// — Guardar todos los libros en el archivo —————  
private void guardarEnArchivo() {  
    try (PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO))) {  
        for (Libro l : libros)  
            pw.println(l.aTexto());  
    } catch (IOException e) {  
        System.out.println("[!] Error al guardar archivo: " + e.getMessage());  
    }  
}
```

// — Encabezado de tabla

```
private void encabezado() {  
    System.out.println("\n" + "-".repeat(70));  
    System.out.printf("%-6s %-30s %-22s %s%n", "COD", "TITULO", "AUTOR",  
"ESTADO");  
    System.out.println("-".repeat(70));  
}
```

```
// — 1. Registrar un nuevo libro —————  
public void registrarLibro(Scanner sc) {  
    System.out.println("\n=== REGISTRAR LIBRO ===");  
    System.out.print("Codigo   : "); int cod = Integer.parseInt(sc.nextLine());  
    System.out.print("Titulo   : "); String tit = sc.nextLine();  
    System.out.print("Autor    : "); String aut = sc.nextLine();  
    System.out.print("Estado (disponible / prestado): "); String est = sc.nextLine();  
  
    // Validar estado  
    if (!est.equals("disponible") && !est.equals("prestado")) {  
        System.out.println("[!] Estado invalido. Se usara 'disponible'.");  
        est = "disponible";  
    }  
  
    libros.add(new Libro(cod, tit, aut, est));  
    guardarEnArchivo();  
    System.out.println("[OK] Libro registrado correctamente.");  
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

// — 2. Buscar libro por título o autor —————

```
public void buscarLibro(Scanner sc) {  
    System.out.println("\n=== BUSCAR LIBRO ===");  
    System.out.print("Ingrese título o autor: ");  
    String termino = sc.nextLine().toLowerCase();  
  
    boolean encontrado = false;  
    encabezado();  
    for (Libro l : libros) {  
        if (l.getTitulo().toLowerCase().contains(termino) ||  
            l.getAutor().toLowerCase().contains(termino)) {  
            l.mostrar();  
            encontrado = true;  
        }  
    }  
    if (!encontrado)  
        System.out.println(" No se encontraron resultados para: " + termino);  
    System.out.println("-".repeat(70));  
}
```

// — 3. Mostrar libros disponibles —————

```
public void mostrarDisponibles() {  
    System.out.println("\n=== LIBROS DISPONIBLES ===");  
    encabezado();  
    long cnt = libros.stream()  
        .filter(l -> l.getEstado().equals("disponible"))  
        .peek(Libro::mostrar)  
        .count();  
    System.out.println("-".repeat(70));  
    System.out.println("Total disponibles: " + cnt);  
}
```

// — 4. Mostrar libros prestados —————

```
public void mostrarPrestados() {  
    System.out.println("\n=== LIBROS PRESTADOS ===");
```



```
encabezado());  
long cnt = libros.stream()  
    .filter(l -> l.getEstado().equals("prestado"))  
    .peek(Libro::mostrar)  
    .count();  
  
System.out.println("-".repeat(70));  
System.out.println("Total prestados: " + cnt);  
}  
  
// — 5. Mostrar todos los libros —————  
public void mostrarTodos() {  
    System.out.println("\n=== TODOS LOS LIBROS ===");  
    encabezado();  
    libros.forEach(Libro::mostrar);  
    System.out.println("-".repeat(70));  
    System.out.println("Total: " + libros.size() + " libro(s)");  
}  
}  
  
// =====  
// CLASE MAIN - Menú principal  
// =====  
public class LibrosVirtual {  
    public static void main(String[] args) {  
        Biblioteca bib = new Biblioteca();  
        Scanner sc = new Scanner(System.in);  
        int opcion;  
  
        do {  
  
            System.out.println("\n ┌──────────────────────────┐");  
            System.out.println(" │ BIBLIOTECA VIRTUAL │");  
            System.out.println(" └──────────────────────────┘");  
            System.out.println(" │ 1. Registrar libro   │");  
            System.out.println(" │ 2. Buscar libro      │");  
            System.out.println(" │ 3. Ver disponibles   │");  
            System.out.println(" │ 4. Ver prestados     │");
```



```
System.out.println("|| 5. Ver todos los libros ||");  
System.out.println("|| 0. Salir ||");  
System.out.println("||_____||");  
System.out.print("Opcion: ");
```

```
opcion = Integer.parseInt(sc.nextLine());
```

```
switch (opcion) {  
    case 1 -> bib.registrarLibro(sc);  
    case 2 -> bib.buscarLibro(sc);  
    case 3 -> bib.mostrarDisponibles();  
    case 4 -> bib.mostrarPrestados();  
    case 5 -> bib.mostrarTodos();  
    case 0 -> System.out.println("Hasta luego!");  
    default-> System.out.println("[!] Opcion invalida.");  
}
```

```
} while (opcion != 0);
```

```
sc.close();
```

```
}
```

```
}
```

2.11.8.9 Prueba de Escritorio

```
--- MENU BIBLIOTECA VIRTUAL ---  
1. Registrar Libro  
2. Buscar Libro  
3. Mostrar Disponibles y En Proceso  
4. Salir  
Seleccione una opcion: 1  
Código: 2378  
Título: POO Introduccion para Principiantes  
Autor: Linus Torvalds  
Estado (1. Terminado / 2. En proceso): 2  
¡Libro registrado con éxito en el archivo!  
  
--- MENU BIBLIOTECA VIRTUAL ---  
1. Registrar Libro  
2. Buscar Libro  
3. Mostrar Disponibles y En Proceso  
4. Salir  
Seleccione una opcion: 1  
Código: 2589  
Título: El Principito  
Autor: Antonie de Saint Exupery  
Estado (1. Terminado / 2. En proceso): 1  
¡Libro registrado con éxito en el archivo!
```



```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 2
Ingrese código o título a buscar: 2378

--- Libro Encontrado ---
Código: 2378 | Título: POO Introduccion para Principiantes | Autor: Linus Torvalds | Estado: En proceso
```

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 3

=== LIBROS TERMINADOS (Disponibles para lectura completa) ===
• Cod: 2589 - El Principito (Antonie de Saint Exupery)

=== LIBROS EN PROCESO (Lectura en curso/Prestados) ===
• Cod: 2378 - POO Introduccion para Principiantes (Linus Torvalds)
```

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 4
PS C:\Users\disba\Documents\Diegui\UTA Primer Semestre\VSCodeProjects>
```

2.11.9 Ejercicio 9

2.11.9.1 Enunciado

Sistema de Ventas

Crear un sistema básico de ventas.

Requerimientos:

- Registrar producto, cantidad, precio y total.
- Generar factura.
- Mostrar historial de ventas.

Aplicar:

- POO
- Métodos
- Archivos

2.11.9.2 Análisis del Problema



El problema requiere el diseño de un sistema interactivo para la gestión y facturación de transacciones comerciales en el entorno de desarrollo PSeInt. El núcleo del problema consiste en simular la persistencia de datos (comportamiento de un archivo físico) y el flujo de la programación orientada a objetos a través de una estructura centralizada basada en arreglos indexados y paralelos. El programa debe capturar de forma interactiva el nombre de un artículo, controlando estrictamente mediante bucles de validación que la cantidad ingresada y el precio unitario sean valores numéricos reales mayores a cero. Tras superar los filtros, el sistema ejecuta de forma automática el cálculo matemático para determinar el importe total de la venta actual (multiplicación de cantidad por precio) y genera un reporte formateado en pantalla que simula una factura física. Para la gestión del historial, el algoritmo debe recorrer secuencialmente las estructuras de almacenamiento interno, imprimiendo las transacciones guardadas y calculando de manera acumulativa indicadores financieros globales, como el número total de operaciones procesadas y los ingresos económicos totales acumulados por el negocio.

2.11.9.3 Declaración de Variables

Variable / Constante	Tipo de Dato	Ámbito / Ubicación	Descripción Funcional
MAX_REGISTROS	Entero	Algoritmo Principal	Constante que define el límite de capacidad de los arreglos (100 registros).
archivoProductos	Cadena	Arreglo Global	Vector indexado que almacena los nombres de los productos vendidos.
archivoCantidades	Entero	Arreglo Global	Vector indexado que guarda las unidades vendidas de cada transacción.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



archivoPrecios	Real	Arreglo Global	Vector indexado que registra los precios unitarios de los productos.
archivoTotales	Real	Arreglo Global	Vector indexado que almacena el importe total calculado de cada venta.
totalRegistros	Entero	Algoritmo Principal	Contador que rastrea la posición actual y la cantidad de filas ocupadas en la memoria.
opcion	Entero	Algoritmo Principal	Almacena la selección del usuario en el menú para el control del bucle Mientras.
contadorVentas	Entero	Reporte Historial	Variable estadística que cuenta cuántas transacciones se despliegan en el reporte.
i	Entero	Reporte Historial	Variable de control utilizada como índice para iterar en el ciclo Para.
prod	Cadena	Captura de Datos	Almacena temporalmente el nombre del producto ingresado por teclado.
cant	Entero	Captura de Datos	Almacena temporalmente la cantidad digitada antes de ser validada y guardada.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



prec	Real	Captura de Datos	Almacena temporalmente el precio unitario digitado antes de ser validado y guardado.
tot	Real	Operación Matemática	Variable temporal que calcula el producto de cant * prec para la factura actual.
granTotalAcumulado	Real	Reporte Historial	Acumulador matemático que calcula la sumatoria monetaria de todas las ventas del historial.
valida	Lógico	Filtros de Control	Bandera de control (Verdadero/Falso) para romper los ciclos de validación de datos.

2.11.9.4 Algoritmo Paso a Paso

Inicio

Establecer el límite máximo de almacenamiento (MAX_REGISTROS <- 100).

Crear y dimensionar los cuatro arreglos paralelos: archivoProductos, archivoCantidades, archivoPrecios y archivoTotales.

Inicializar el contador totalRegistros en 0 y la variable opcion en 0.

Ciclo Principal (Menú Interactivo):

Mostrar en pantalla las opciones: 1. Registrar Nueva Venta (Facturar), 2. Mostrar Historial de Ventas, 3. Salir del programa.

Leer la opcion ingresada por el usuario.

Si la opción es 1 (Registrar Venta):



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Validar si hay espacio libre ($\text{totalRegistros} < \text{MAX_REGISTROS}$). Si está lleno, mostrar mensaje de error.

Si hay espacio, incrementar totalRegistros en 1 para apuntar a la nueva posición del arreglo.

Solicitar y leer el nombre del producto (prod) y guardarlo en $\text{archivoProductos}[\text{totalRegistros}]$.

Ciclo Validar Cantidad: Solicitar la cantidad (cant). Si es menor o igual a 0, mostrar error y volver a pedirla. Al ser válida, guardarla en $\text{archivoCantidades}[\text{totalRegistros}]$.

Ciclo Validar Precio: Solicitar el precio (prec). Si es menor o igual a 0, mostrar error y volver a pedirlo. Al ser válido, guardarlo en $\text{archivoPrecios}[\text{totalRegistros}]$.

Calcular automáticamente el importe multiplicando los datos validados ($\text{tot} < \text{cant} * \text{prec}$) y almacenarlo en $\text{archivoTotales}[\text{totalRegistros}]$.

Mostrar en pantalla el diseño de la Factura de Venta imprimiendo de forma ordenada el producto, la cantidad, el precio unitario y el total calculado.

Si la opción es 2 (Mostrar Historial):

Inicializar las variables estadísticas contadorVentas y $\text{granTotalAcumulado}$ en 0.

Mostrar la cabecera del reporte del historial.

Verificar Contenido: Si totalRegistros es igual a 0, mostrar un aviso indicando que aún no hay ventas guardadas.

Si existen datos, ejecutar un Ciclo Desde/Para controlado por la variable i desde 1 hasta el valor de totalRegistros :

Imprimir en una sola fila los datos de la posición actual:
 $\text{archivoProductos}[i]$, $\text{archivoCantidades}[i]$, $\text{archivoPrecios}[i]$ y
 $\text{archivoTotales}[i]$.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Sumar el valor de `archivoTotales[i]` al acumulador
`granTotalAcumulado`.

Incrementar el contador `Ventas` en 1.

Mostrar en la base del reporte el balance estadístico final con el total de ventas
procesadas y los ingresos acumulados.

Si la opción es 3 (Salir):

Mostrar un mensaje de despedida en pantalla.

De Otro Modo:

Informar al usuario que la opción no es válida.

Repetir el Menú de Operaciones mientras la opcion sea diferente de 3.

Fin

2.11.9.5 Pseudocódigo

Algoritmo Sistema_Ventas

Definir `MAX_REGISTROS` Como Entero

`MAX_REGISTROS` <- 100

Definir `archivoNombres` Como Cadena

Definir `archivoCantidades` Como Entero

Definir `archivoPrecios`, `archivoTotales` Como Real

Dimension `archivoNombres`[`MAX_REGISTROS`]

Dimension `archivoCantidades`[`MAX_REGISTROS`]

Dimension `archivoPrecios`[`MAX_REGISTROS`]

Dimension `archivoTotales`[`MAX_REGISTROS`]

Definir `totalVentas`, `opcion` Como Entero

`totalVentas` <- 0

Repetir



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir ""

Escribir "=====

Escribir " SISTEMA DE VENTAS "

Escribir "=====

Escribir "1. Registrar Producto (Venta)"

Escribir "2. Generar Factura Actual"

Escribir "3. Mostrar Historial de Ventas"

Escribir "4. Salir del Sistema"

Escribir "-----"

Escribir Sin Saltar "Seleccione una opcion (1-4): "

Leer opcion

Segun opcion Hacer

1:

RegistrarVenta(archivoNombres, archivoCantidades,
archivoPrecios, archivoTotales, totalVentas, MAX_REGISTROS)

2:

GenerarFactura(archivoNombres, archivoCantidades,
archivoPrecios, archivoTotales, totalVentas)

3:

MostrarHistorial(archivoNombres, archivoCantidades,
archivoPrecios, archivoTotales, totalVentas)

4:

Escribir "Cerrando el sistema de ventas... ¡Hasta
luego!"

De Otro Modo:

Escribir "[Error] Opcion invalida. Intente de nuevo."

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

SubProceso RegistrarVenta(nombres, cantidades, precios, totales, totalVentas Por
Referencia, MAX_REGISTROS)

Definir prod Como Cadena

Definir cant Como Entero

Definir prec, tot Como Real



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Si totalVentas < MAX_REGISTROS Entonces

totalVentas <- totalVentas + 1

Escribir ""

Escribir "--- REGISTRAR NUEVA VENTA ---"

Escribir Sin Saltar "Nombre del producto: "

Leer prod

Escribir Sin Saltar "Cantidad: "

Leer cant

Escribir Sin Saltar "Precio unitario: \$"

Leer prec

tot <- cant * prec

nombres[totalVentas] <- prod

cantidades[totalVentas] <- cant

precios[totalVentas] <- prec

totales[totalVentas] <- tot

Escribir ""

Escribir ">> Venta registrada con exito. Total: \$", tot

Sino

Escribir "[Error] Archivo lleno. No se admiten mas registros."

FinSi

FinSubProceso

SubProceso GenerarFactura(nombres, cantidades, precios, totales, totalVentas)

Si totalVentas = 0 Entonces

Escribir "[Aviso] No hay datos en el sistema para generar una factura."

Sino

Escribir ""

Escribir "=====

Escribir " FACTURA DE VENTA "

Escribir "=====

Escribir "Producto: ", nombres[totalVentas]

Escribir "Cantidad: ", cantidades[totalVentas]

Escribir "Precio Unit: \$", precios[totalVentas]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```

    Escribir "-----"
    Escribir "TOTAL A PAGAR: $", totales[totalVentas]
    Escribir "=====
FinSi
FinSubProceso

SubProceso MostrarHistorial(nombres, cantidades, precios, totales, totalVentas)
    Definir i Como Entero
    Definir acumulado Como Real
    acumulado <- 0

    Escribir ""
    Escribir
    "=====
    Escribir "          HISTORIAL DE VENTAS (ARCHIVO)          "
    Escribir
    "=====

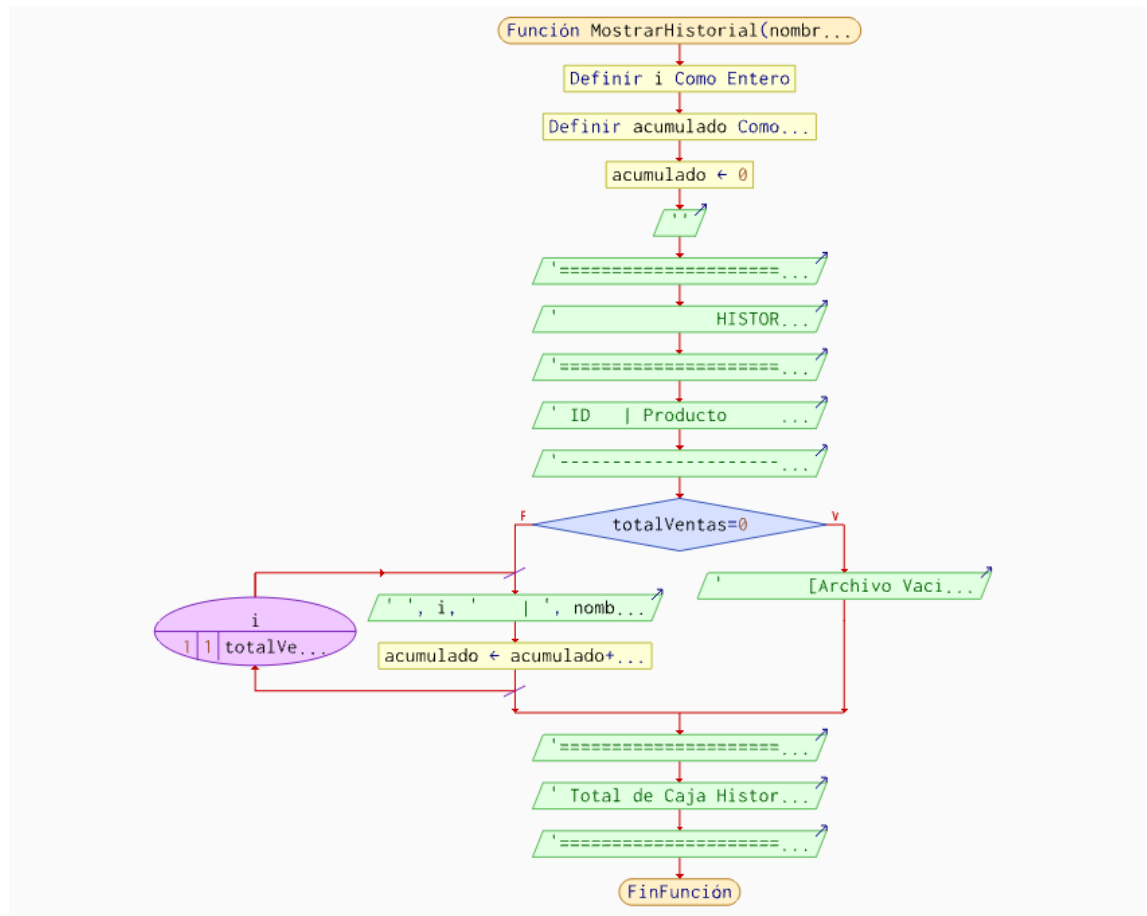
    Escribir " ID   | Producto       | Cant. | Precio U. | Total   "
    Escribir "-----"

    Si totalVentas = 0 Entonces
        Escribir "          [Archivo Vacio] No hay ventas guardadas.          "
    Sino
        Para i <- 1 Hasta totalVentas Con Paso 1 Hacer
            Escribir " ", i, "   | ", nombres[i], "          | ", cantidades[i], "   |
$, precios[i], "   | $", totales[i]
            acumulado <- acumulado + totales[i]
        FinPara
    FinSi
    Escribir
    "=====
    Escribir " Total de Caja Historico: $", acumulado
    Escribir
    "=====
FinSubProceso
```

2.11.9.6 Diagrama de flujo

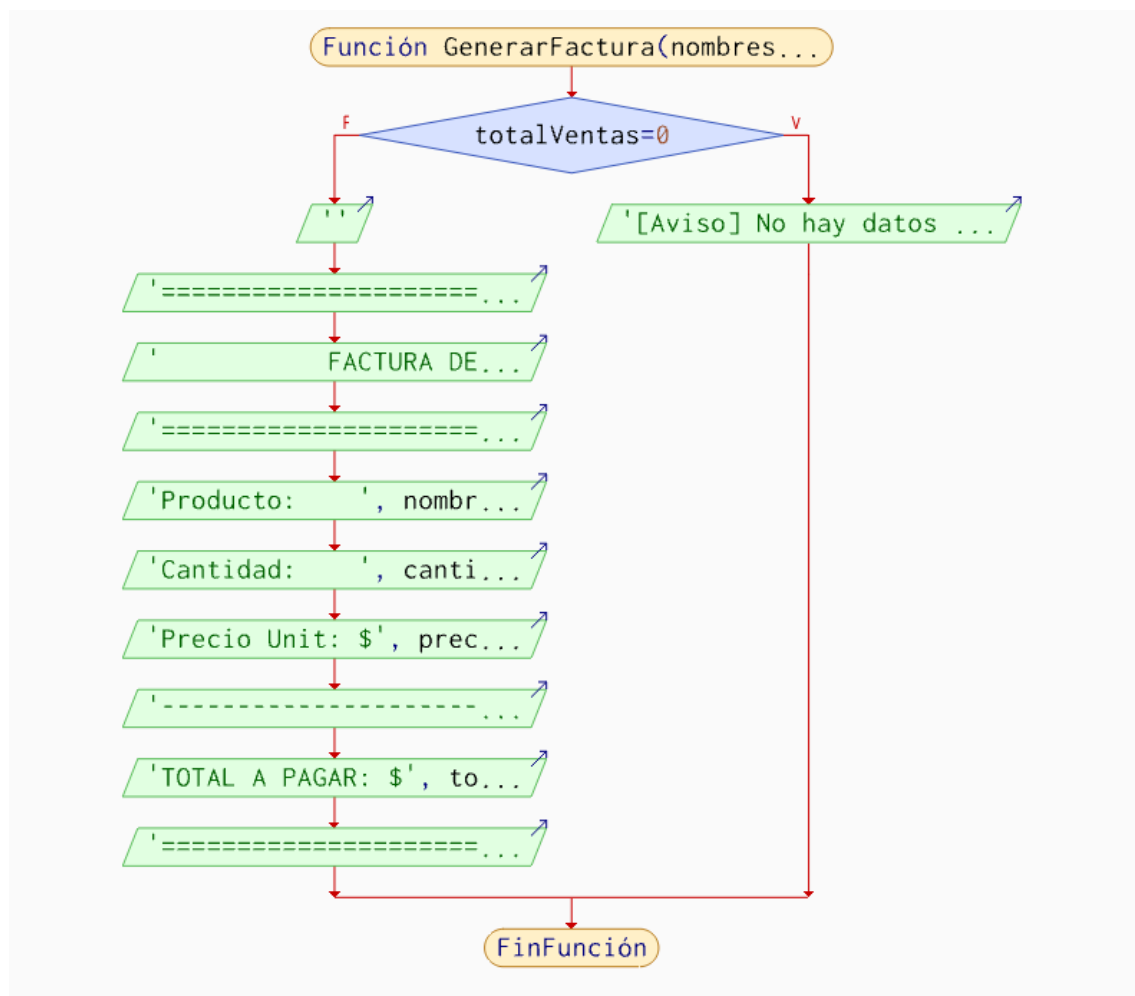


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



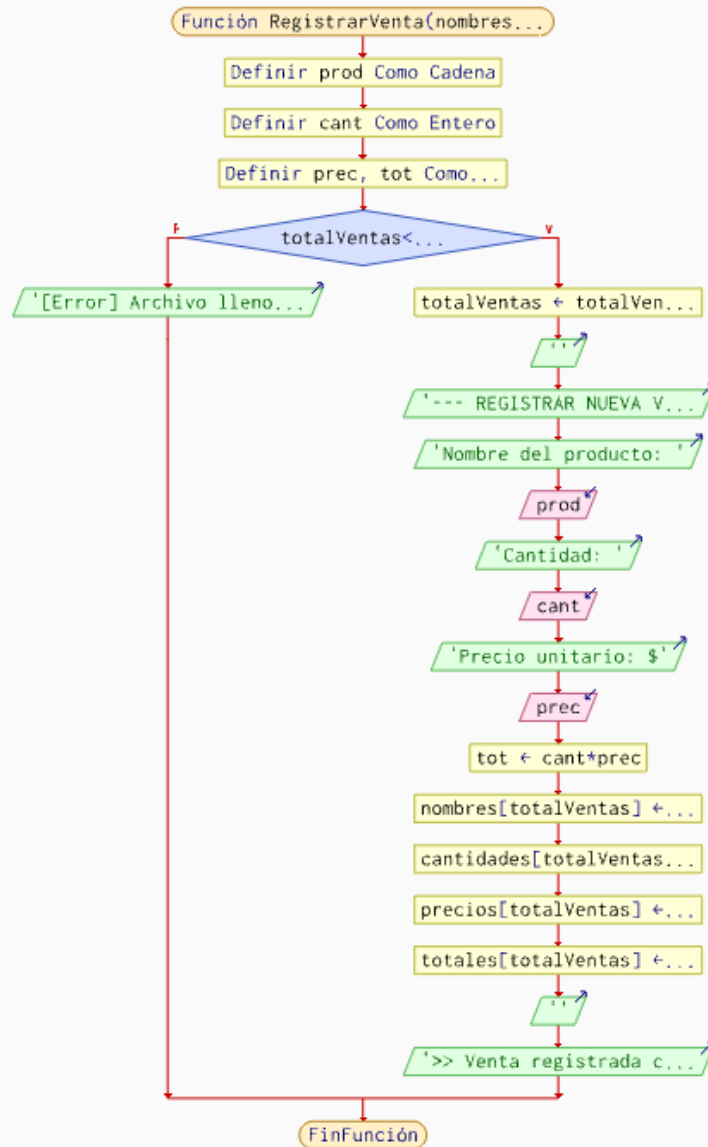


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



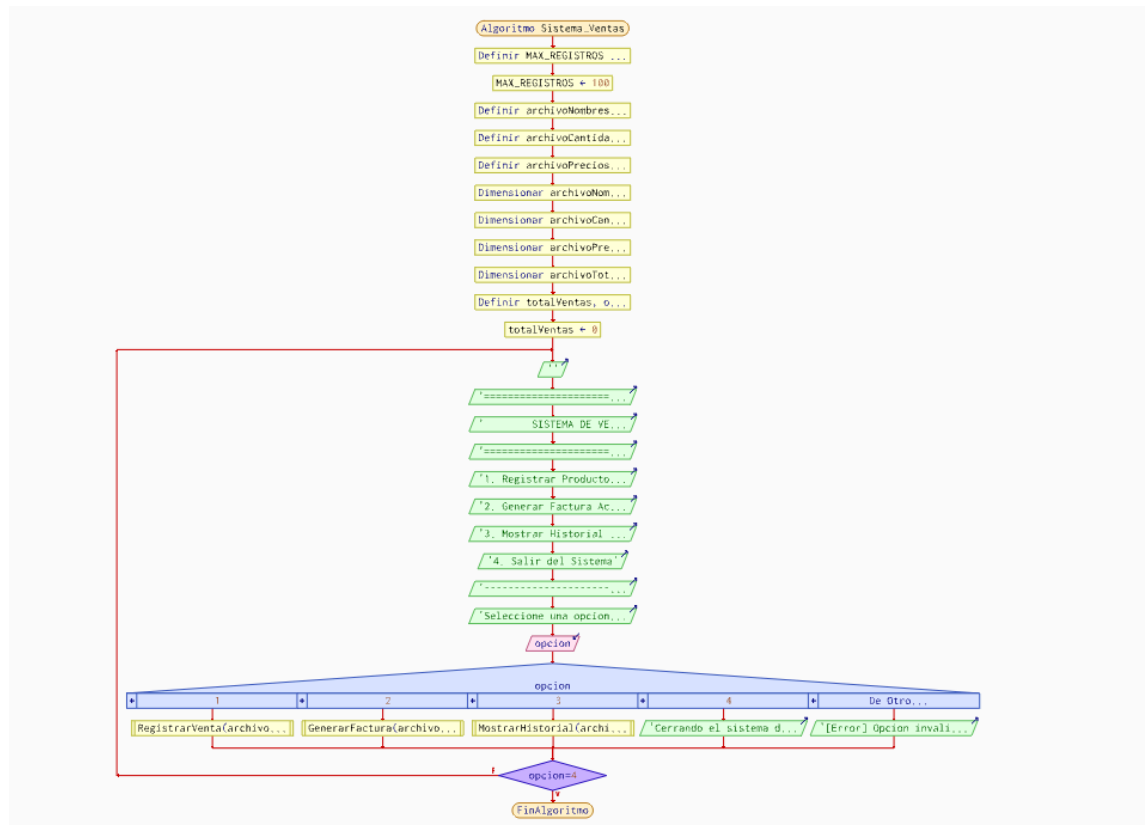


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



2.11.9.7 Codificación C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
#include <sstream>
```

```
using namespace std;
```

```
//
```

```
=====
```

```
=====
```

```
// CLASE VENTA (Aplicación de POO)
```

```
//
```

```
=====
```

```
=====
```

```
class Venta {
```

```
private:
```

```
    // Atributos encapsulados
```



string producto;

int cantidad;

double precio;

double total;

public:

// Constructor por defecto

Venta() {

 producto = "";

 cantidad = 0;

 precio = 0.0;

 total = 0.0;

}

// Métodos Setters y Getters (Métodos de acceso)

void setProducto(string prod) { producto = prod; }

void setCantidad(int cant) { cantidad = cant; }

void setPrecio(double prec) { precio = prec; }

string getProducto() { return producto; }

int getCantidad() { return cantidad; }

double getPrecio() { return precio; }

double getTotal() { return total; }

// Método para calcular el total automáticamente

void calcularTotal() {

 total = cantidad * precio;

}

// REQUERIMIENTO: Guardar el registro en el archivo de texto

void guardarEnArchivo() {

 // Modo ios::app para añadir datos al final sin borrar lo existente

 ofstream archivoEscritura("ventas.txt", ios::app);

 if (!archivoEscritura.is_open()) {

 cout << " [Error] No se pudo abrir el archivo para registrar la venta.\n";

 return;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
}

// Guardamos los atributos separados por el delimitador '|'
archivoEscritura << producto << "|" << cantidad << "|" << precio << "|" << total
<< "\n";
archivoEscritura.close();
}

// REQUERIMIENTO: Mostrar la factura de la venta actual en pantalla
void generarFactura() {
    cout << "\n===== \n";
    cout << "      FACTURA DE VENTA      \n";
    cout << "===== \n";
    cout << " Producto: " << producto << "\n";
    cout << " Cantidad: " << cantidad << "\n";
    cout << " Precio Unit:" << fixed << setprecision(2) << " $" << precio << "\n";
    cout << " ----- \n";
    cout << " TOTAL: " << fixed << setprecision(2) << " $" << total << "\n";
    cout << "===== \n";
}
};

//
=====
=====

// FUNCIÓN GLOBAL: HISTORIAL DE VENTAS (Manejo de Archivos)
//
=====
=====

void mostrarHistorialVentas() {
    ifstream archivoLectura("ventas.txt");

    if (!archivoLectura.is_open()) {
        cout << "\n[Aviso] No existen ventas registradas en el historial (Archivo no
encontrado).\n";
        return;
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
string linea;
double granTotalAcumulado = 0.0;
int contadorVentas = 0;

cout <<
"\n=====
=====\\n";
cout << "          HISTORIAL GENERAL DE VENTAS          \\n";
cout <<
"=====
=====\\n";
cout << " " << left << setw(20) << "Producto"
    << setw(12) << "Cantidad"
    << setw(15) << "Precio Unit."
    << setw(15) << "Total Venta" << "\\n";
cout << " -----\\n";

// Ciclo para leer el archivo línea por línea
while (getline(archivoLectura, linea)) {
    if (linea.empty()) continue;

    stringstream ss(linea);
    string prod, cantStr, precStr, totStr;

    // Descomposición de la línea usando el delimitador '|'
    getline(ss, prod, '|');
    getline(ss, cantStr, '|');
    getline(ss, precStr, '|');
    getline(ss, totStr, '|');

    // Conversión de las cadenas leídas a sus tipos de datos correspondientes
    int cant = stoi(cantStr);
    double prec = stod(precStr);
    double tot = stod(totStr);

    granTotalAcumulado += tot;
```



```
contadorVentas++;
```

```
// Imprimir la fila con formato alineado
```

```
cout << " " << left << setw(20) << prod
```

```
    << setw(12) << cant
```

```
    << " $" << setw(13) << fixed << setprecision(2) << prec
```

```
    << " $" << setw(13) << fixed << setprecision(2) << tot << "\n";
```

```
}
```

```
archivoLectura.close();
```

```
cout <<
```

```
"=====
=====\\n";
```

```
cout << " Total de ventas procesadas: " << contadorVentas << "\\n";
```

```
cout << " INGRESOS TOTALES ACUMULADOS: $" << fixed << setprecision(2)
<< granTotalAcumulado << "\\n";
```

```
cout <<
```

```
"=====
=====\\n";
```

```
}
```

```
//
```

```
=====
=====
```

```
// PROGRAMA PRINCIPAL
```

```
//
```

```
=====
=====
```

```
int main() {
```

```
    int opcion;
```

```
    do {
```

```
        cout << "\\n=====\\n";
```

```
        cout << "        SISTEMA DE VENTAS        \\n";
```

```
        cout << "=====\\n";
```

```
        cout << "1. Registrar Nueva Venta (Facturar)\\n";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "2. Mostrar Historial de Ventas\n";
cout << "3. Salir del programa\n";
cout << "-----\n";
cout << "Seleccione una opcion (1-3): ";

// Validación para evitar bucles infinitos con caracteres inválidos
if (!(cin >> opcion)) {
    cout << "[Error] Entrada invalida. Intente de nuevo.\n";
    cin.clear();
    cin.ignore(10000, '\n');
    continue;
}

switch (opcion) {
    case 1: {
        Venta nuevaVenta; // Instanciación del objeto
        string prod;
        int cant;
        double prec;

        cout << "\n--- INGRESO DE DATOS DE LA VENTA ---\n";
        cout << " Nombre del producto: ";
        cin.ignore(); // Limpia el buffer
        getline(cin, prod);
        nuevaVenta.setProducto(prod);

        // Validación de cantidad
        do {
            cout << " Cantidad: ";
            if (cin >> cant && cant > 0) break;
            cout << " [Error] Cantidad debe ser un entero mayor a 0.\n";
            cin.clear(); cin.ignore(10000, '\n');
        } while (true);
        nuevaVenta.setCantidad(cant);

        // Validacion de precio
        do {
```



```
        cout << " Precio unitario: ";
        if (cin >> prec && prec > 0.0) break;
        cout << " [Error] El precio debe ser un numero mayor a 0.\n";
        cin.clear(); cin.ignore(10000, '\n');
    } while (true);
    nuevaVenta.setPrecio(prec);

    // Procesamiento mediante métodos del objeto
    nuevaVenta.calcularTotal();
    nuevaVenta.guardarEnArchivo();
    nuevaVenta.generarFactura();
    break;
}
case 2:
    mostrarHistorialVentas();
    break;
case 3:
    cout << "\nSaliendo del sistema de ventas... ¡Hasta luego!\n";
    break;
default:
    cout << "[Error] La opcion seleccionada no existe.\n";
}
} while (opcion != 3);

return 0;
}
```

2.11.9.8 Codificación Java

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



//

=====

=====

// CLASE VENTA (Aplicación estricta de POO)

//

=====

=====

```
class Venta {  
    // Atributos privados (Encapsulamiento)  
    private String producto;  
    private int cantidad;  
    private double precio;  
    private double total;  
  
    // Constructor por defecto  
    public Venta() {  
        this.producto = "";  
        this.cantidad = 0;  
        this.precio = 0.0;  
        this.total = 0.0;  
    }  
  
    // Métodos Setters y Getters  
    public void setProducto(String producto) { this.producto = producto; }  
    public void setCantidad(int cantidad) { this.cantidad = cantidad; }  
    public void setPrecio(double precio) { this.precio = precio; }  
  
    public String getProducto() { return producto; }  
    public int getCantidad() { return cantidad; }  
    public double getPrecio() { return precio; }  
    public double getTotal() { return total; }  
  
    // Método para calcular el total de forma autónoma  
    public void calcularTotal() {  
        this.total = this.cantidad * this.precio;  
    }  
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// REQUERIMIENTO: Guardar el registro actual en el archivo de texto
public void guardarEnArchivo(String nombreArchivo) {
    // Habilitamos el modo append (true) para añadir filas al final del archivo
    try (FileWriter fw = new FileWriter(nombreArchivo, true);
        PrintWriter archivoEscritura = new PrintWriter(fw)) {

        // Guardamos los atributos separados por el delimitador '|'
        archivoEscritura.println(producto + "|" + cantidad + "|" + precio + "|" + total);

    } catch (IOException e) {
        System.out.println(" [Error] No se pudo escribir en el archivo: " +
            e.getMessage());
    }
}

// REQUERIMIENTO: Mostrar la factura en pantalla
public void generarFactura() {
    System.out.println("\n=====");
    System.out.println("      FACTURA DE VENTA      ");
    System.out.println("=====");
    System.out.println(" Producto: " + producto);
    System.out.println(" Cantidad: " + cantidad);
    System.out.printf(" Precio Unit: $%.2f\n", precio);
    System.out.println(" -----");
    System.out.printf(" TOTAL:    $%.2f\n", total);
    System.out.println("=====");
}

//
=====
=====

// CLASE PRINCIPAL: SistemaVentas
//
=====
=====

public class SistemaVentas {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
private static final String NOMBRE_ARCHIVO = "ventas.txt";
private static final Scanner teclado = new Scanner(System.in);

// REQUERIMIENTO: Mostrar historial leyendo desde el archivo
public static void mostrarHistorialVentas() {
    File archivo = new File(NOMBRE_ARCHIVO);

    if (!archivo.exists()) {
        System.out.println("\n[Aviso] No existen ventas registradas en el historial.");
        return;
    }

    double granTotalAcumulado = 0.0;
    int contadorVentas = 0;

    System.out.println("\n=====
=====");
    System.out.println("                                HISTORIAL GENERAL DE VENTAS
");

    System.out.println("=====
=====");
    System.out.printf("  %-20s  %-12s  %-15s  %-15s\n", "Producto", "Cantidad",
"Precio Unit.", "Total Venta");
    System.out.println(" -----");

    try (FileReader fr = new FileReader(archivo);
        BufferedReader archivoLectura = new BufferedReader(fr)) {

        String linea;
        while ((linea = archivoLectura.readLine()) != null) {
            if (linea.trim().isEmpty()) continue;

            // Segmentamos la línea usando la barra vertical como expresión regular
            String[] partes = linea.split("\\|");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        if (partes.length == 4) {
            String prod = partes[0];
            int cant = Integer.parseInt(partes[1]);
            double prec = Double.parseDouble(partes[2]);
            double tot = Double.parseDouble(partes[3]);

            granTotalAcumulado += tot;
            contadorVentas++;

            // Mostrar fila con formato ordenado
            System.out.printf(" %-20s %-12d $%-14.2f $%-14.2f\n", prod, cant, prec,
tot);
        }
    }

    System.out.println("=====
=====");
    System.out.println(" Total de ventas procesadas: " + contadorVentas);
    System.out.printf(" INGRESOS TOTALES ACUMULADOS: $%.2f\n",
granTotalAcumulado);

    System.out.println("=====
=====");

    } catch (IOException | NumberFormatException e) {
        System.out.println("Error al procesar el archivo de historial: " +
e.getMessage());
    }
}

// PROGRAMA PRINCIPAL
public static void main(String[] args) {
    int opcion = 0;

    do {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("\n=====");
System.out.println("    SISTEMA DE VENTAS    ");
System.out.println("=====");
System.out.println("1. Registrar Nueva Venta (Facturar)");
System.out.println("2. Mostrar Historial de Ventas");
System.out.println("3. Salir del programa");
System.out.println("-----");
System.out.print("Seleccione una opcion (1-3): ");

if (teclado.hasNextInt()) {
    opcion = teclado.nextInt();

    switch (opcion) {
        case 1:
            Venta nuevaVenta = new Venta(); // Instanciación del objeto

            System.out.println("\n--- INGRESO DE DATOS DE LA VENTA ---");
            System.out.print(" Nombre del producto: ");
            teclado.nextLine(); // Limpieza del buffer del Scanner
            String prod = teclado.nextLine();
            nuevaVenta.setProducto(prod);

            // Validación de cantidad mayor a cero
            int cant;
            while (true) {
                System.out.print(" Cantidad: ");
                if (teclado.hasNextInt()) {
                    cant = teclado.nextInt();
                    if (cant > 0) break;
                } else {
                    teclado.next(); // Descartar caracter erróneo
                }
                System.out.println(" [Error] Cantidad debe ser un entero mayor a 0.");
            }
            nuevaVenta.setCantidad(cant);

            // Validación de precio mayor a cero
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
double prec;
while (true) {
    System.out.print(" Precio unitario: ");
    if (teclado.hasNextDouble()) {
        prec = teclado.nextDouble();
        if (prec > 0.0) break;
    } else {
        teclado.next(); // Descartar caracter erróneo
    }
    System.out.println(" [Error] El precio debe ser un numero mayor a
0.");
}
nuevaVenta.setPrecio(prec);

// Ejecución de la lógica interna mediante métodos del objeto
nuevaVenta.calcularTotal();
nuevaVenta.guardarEnArchivo(NOMBRE_ARCHIVO);
nuevaVenta.generarFactura();
break;

case 2:
    mostrarHistorialVentas();
    break;

case 3:
    System.out.println("\nSaliendo del sistema de ventas... ¡Hasta luego!");
    break;

default:
    System.out.println("[Error] La opcion seleccionada no existe.");
}
} else {
    System.out.println("[Error] Entrada no valida. Debe ingresar un numero.");
    teclado.next(); // Limpieza del token inválido
    opcion = 0;
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
} while (opcion != 3);
```

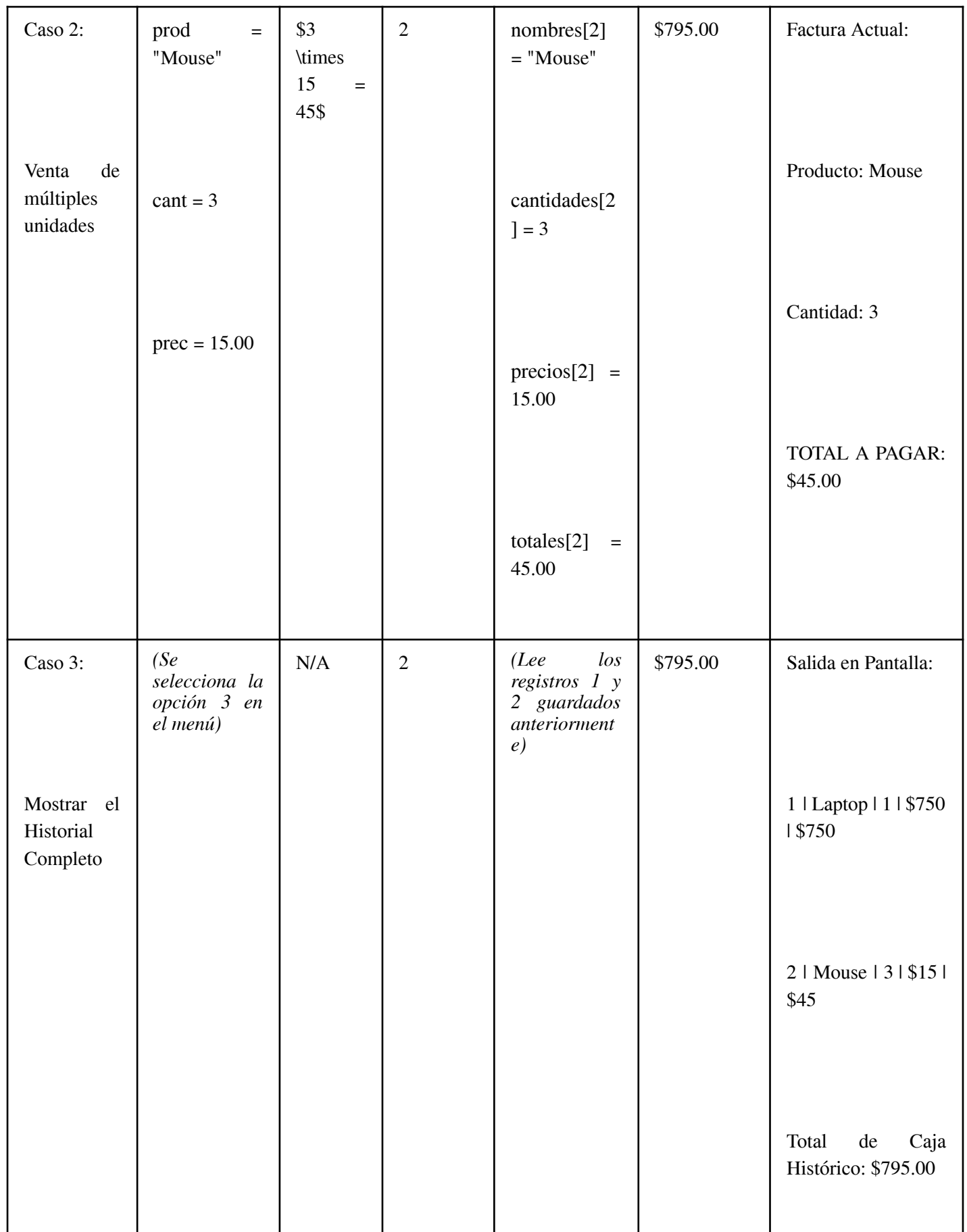
```
teclado.close();
```

```
}
```

```
}
```

2.11.9.9 Prueba de Escritorio

Caso / Acción	Entrada del Usuario (prod, cant, prec)	Lógica / Cálculo (tot = cant * prec)	totalVent as (Índice)	Datos Almacenado s en "Archivo"	acumulado (Caja Histórica)	Resultado en Pantalla / Factura
Caso 1: Venta de un producto unitario	prod = "Laptop" cant = 1 prec = 750.00	\$1 \times 750 = 750\$	1	nombres[1] = "Laptop" cantidades[1]] = 1 precios[1] = 750.00 totales[1] = 750.00	\$750.00	Factura Actual: Producto: Laptop Cantidad: 1 TOTAL A PAGAR: \$750.00





2.11.10 Ejercicio 10

2.11.10.1 Enunciado

Mini Proyecto Integrador

Desarrollar un sistema completo aplicando POO y archivos.

Requerimientos:

- Implementar CRUD.
- Persistencia en archivos.
- Menú interactivo y validaciones.

Aplicar:

- POO
- Archivos
- Métodos
- Validaciones

2.11.10.2 Análisis del Problema

Se requiere desarrollar un sistema que permita gestionar una agenda de contactos utilizando programación orientada a objetos y almacenamiento en archivos de texto. El sistema debe permitir registrar, consultar, buscar y eliminar contactos, garantizando la persistencia de los datos mediante archivos. Además, debe incluir un menú interactivo que permita al usuario seleccionar las diferentes opciones, así como validaciones básicas para asegurar el correcto ingreso de datos.

2.11.10.3 Declaración de Variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
nombre	String	String	Almacena el nombre del contacto.
teléfono	String	String	Guarda el número telefónico del contacto.
correo	String	String	Almacena el correo electrónico del contacto.
linea	String	String	Guarda temporalmente cada línea leída del archivo.



opcion	int	int	Permite seleccionar una opción del menú.
encontrado	bool	boolean	Indica si el contacto fue encontrado en la búsqueda.
archivo	String	String	Representa el nombre del archivo donde se almacenan los datos (agenda.txt).

2.11.10.4 Algoritmo Paso a Paso

Iniciar programa

Mostrar menú de opciones

Leer opción del usuario

Si opción = 1 → Agregar contacto

Si opción = 2 → Mostrar contactos

Si opción = 3 → Buscar contacto

Si opción = 4 → Eliminar contacto

Si opción = 5 → Salir

Repetir hasta que el usuario seleccione salir

2.11.10.5 Pseudocódigo

Algoritmo AgendaCRUD

Definir opcion Como Entero

Repetir

 Escribir "1. Agregar"

 Escribir "2. Mostrar"

 Escribir "3. Buscar"

 Escribir "4. Eliminar"

 Escribir "5. Salir"

 Leer opcion

Segun opcion Hacer

 1:

 Escribir "Agregar contacto"



2:

Escribir "Mostrar contactos"

3:

Escribir "Buscar contacto"

4:

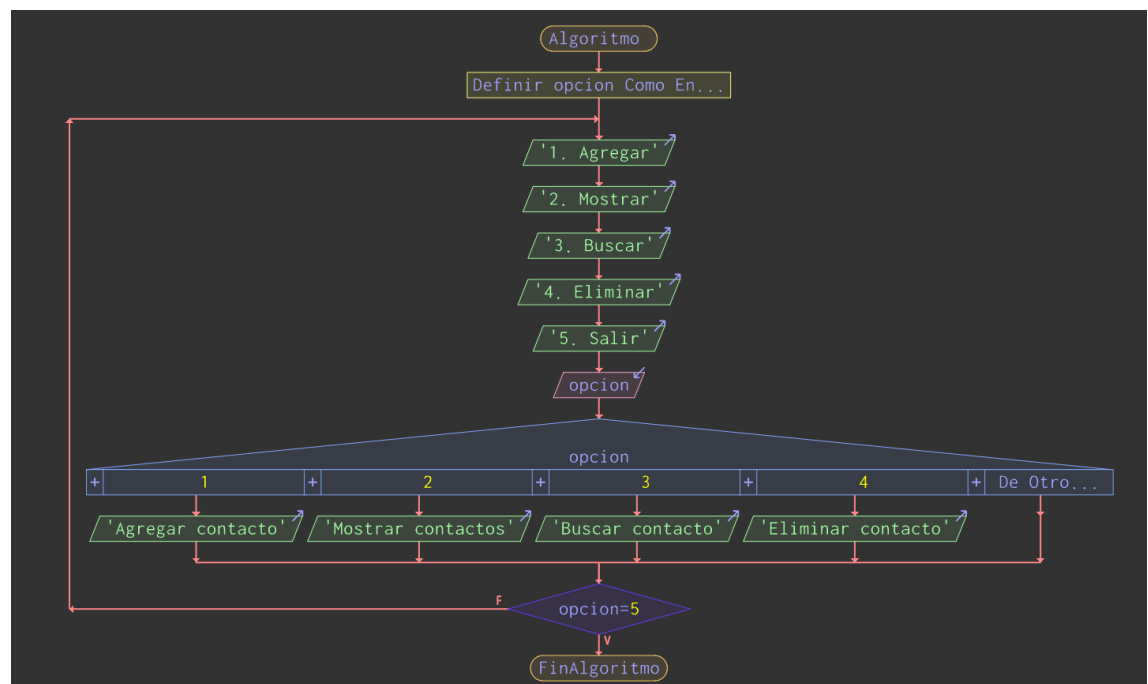
Escribir "Eliminar contacto"

FinSegun

Hasta Que opcion = 5

FinAlgoritmo

2.11.10.6 Diagrama de flujo



2.11.10.7 Codificación C++

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// CLASE
class Contacto {
public:
    string nombre, telefono, correo;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Contacto(string n, string t, string c) {
    nombre = n;
    telefono = t;
    correo = c;
}

string toString() {
    return nombre + "," + telefono + "," + correo;
}

};

string archivo = "agenda.txt";

//MÉTODOS
void agregar() {
    ofstream file(archivo, ios::app);
    string n, t, c;

    cout << "Nombre: ";
    getline(cin, n);
    cout << "Telefono: ";
    getline(cin, t);
    cout << "Correo: ";
    getline(cin, c);

    Contacto con(n, t, c);
    file << con.toString() << endl;

    file.close();
}

void mostrar() {
    ifstream file(archivo);
    string linea;

    while (getline(file, linea)) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        cout << linea << endl;
    }

    file.close();
}

void buscar() {
    ifstream file(archivo);
    string linea, nombre;
    bool encontrado = false;

    cout << "Nombre a buscar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) != string::npos) {
            cout << "Encontrado: " << linea << endl;
            encontrado = true;
        }
    }

    if (!encontrado)
        cout << "No encontrado\n";

    file.close();
}

void eliminar() {
    ifstream file(archivo);
    ofstream temp("temp.txt");
    string linea, nombre;

    cout << "Nombre a eliminar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) == string::npos) {
```



```
        temp << linea << endl;
    }
}

file.close();
temp.close();

remove("agenda.txt");
rename("temp.txt", "agenda.txt");

cout << "Contacto eliminado\n";
}

int main() {
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Buscar\n4. Eliminar\n5. Salir\n";
        cout << "Opcion: ";
        cin >> op;
        cin.ignore();

        switch (op) {
            case 1: agregar(); break;
            case 2: mostrar(); break;
            case 3: buscar(); break;
            case 4: eliminar(); break;
        }

    } while (op != 5);

    return 0;
}
```

2.11.10.8 Codificación Java

```
// Librerías necesarias
import java.io.*;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
import java.util.Scanner;

// CLASE
class Contacto {

    // Atributos
    String nombre;
    String telefono;
    String correo;

    // Constructor
    public Contacto(String n, String t, String c) {

        nombre = n;
        telefono = t;
        correo = c;
    }

    // Método para convertir los datos en texto
    public String toString() {

        return nombre + "," + telefono + "," + correo;
    }
}

// Clase principal
public class Main {

    // Nombre del archivo
    static String archivo = "agenda.txt";

    // MÉTODO AGREGAR
    public static void agregar(Scanner entrada) {

        try {

            // Abrir archivo en modo agregar
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
FileWriter file = new FileWriter(archivo, true);

String n, t, c;

System.out.print("Nombre: ");
n = entrada.nextLine();

System.out.print("Telefono: ");
t = entrada.nextLine();

System.out.print("Correo: ");
c = entrada.nextLine();

// Crear objeto
Contacto con = new Contacto(n, t, c);

// Guardar en archivo
file.write(con.toString() + "\n");

file.close();

System.out.println("Contacto guardado");

} catch(IOException e) {

    System.out.println("Error al guardar");
}

}

// MÉTODO MOSTRAR
public static void mostrar() {

    try {

        BufferedReader file = new BufferedReader(new FileReader(archivo));

        String linea;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Leer línea por línea
while((linea = file.readLine()) != null) {

    System.out.println(linea);
}

file.close();

} catch(IOException e) {

    System.out.println("Error al leer el archivo");
}
}

// MÉTODO BUSCAR
public static void buscar(Scanner entrada) {

    try {

        BufferedReader file = new BufferedReader(new FileReader(archivo));

        String linea;
        String nombre;

        boolean encontrado = false;

        System.out.print("Nombre a buscar: ");
        nombre = entrada.nextLine();

        // Buscar contacto
        while((linea = file.readLine()) != null) {

            if(linea.contains(nombre)) {

                System.out.println("Encontrado: " + linea);
                encontrado = true;
            }
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
    }  
}  
  
// Verificar si no existe  
if(!encontrado) {  
  
    System.out.println("No encontrado");  
}  
  
file.close();  
  
} catch(IOException e) {  
  
    System.out.println("Error al buscar");  
}  
}  
  
// MÉTODO ELIMINAR  
public static void eliminar(Scanner entrada) {  
  
    try {  
  
        BufferedReader file = new BufferedReader(new FileReader(archivo));  
  
        FileWriter temp = new FileWriter("temp.txt");  
  
        String linea;  
        String nombre;  
  
        System.out.print("Nombre a eliminar: ");  
        nombre = entrada.nextLine();  
  
        // Copiar solo los contactos diferentes  
        while((linea = file.readLine()) != null) {  
  
            if(!linea.contains(nombre)) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        temp.write(linea + "\n");
    }
}

file.close();
temp.close();

// Eliminar archivo original
File original = new File("agenda.txt");
original.delete();

// Renombrar temporal
File nuevo = new File("temp.txt");
nuevo.renameTo(original);

System.out.println("Contacto eliminado");

} catch(IOException e) {

    System.out.println("Error al eliminar");
}
}

// MÉTODO PRINCIPAL
public static void main(String[] args) {

    Scanner entrada = new Scanner(System.in);

    int op;

    do {

        // Menú
        System.out.println("\n1. Agregar");
        System.out.println("2. Mostrar");
        System.out.println("3. Buscar");
        System.out.println("4. Eliminar");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("5. Salir");
```

```
System.out.print("Opcion: ");
```

```
op = entrada.nextInt();
```

```
entrada.nextLine();
```

```
// Opciones
```

```
switch(op) {
```

```
    case 1:
```

```
        agregar(entrada);
```

```
        break;
```

```
    case 2:
```

```
        mostrar();
```

```
        break;
```

```
    case 3:
```

```
        buscar(entrada);
```

```
        break;
```

```
    case 4:
```

```
        eliminar(entrada);
```

```
        break;
```

```
    }
```

```
} while(op != 5);
```

```
entrada.close();
```

```
}
```

```
}
```

2.11.10.9 Prueba de Escritorio



```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 1
Nombre: Nancy
Telefono: 0997745824
Correo: nancyp@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 2
Michael,0979350478,mruiz3405@gmail.com
Michael,0979350748,msanchez3405@gmail.com
Nancy,0997745824,nancyp@gmail.com
```

```
3. Buscar
4. Eliminar
5. Salir
Opcion: 3
Nombre a buscar: Nancy
Encontrado: Nancy,0997745824,nancyp@gmail.com

1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 4
Nombre a eliminar: Nancy
Contacto eliminado
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 5
PS C:\Users\HP\Documents\ALGORITMOS\Agenda>
```

2.12 Link del GitHub

https://github.com/TurKellou/APE-05_Creaci-n_Lectura_y_Escritura_de_Ficheros.git